

Orbit Formation

Construire, vérifier et enseigner avec son assistant personnel

Jean-Sébastien Beaulieu

Table des matières

Orbit Formation	5
Construire, vérifier et enseigner avec son assistant personnel	5
Note d'édition et de responsabilité	5
Collaboration — déclaration à examiner lors de la revue d'auteur	5
À qui s'adresse le manuel ?	5
Comment l'utiliser	5
Table des matières	6
Le parcours de 40 heures	7
Le rôle de chacun	7
Trois destinations à comprendre	7
Un notebook gratuit et une reprise explicite	8
Partie I — comprendre, expérimenter et construire	9
Module 1 — Geste, état et rendu	10
Avant de commencer	10
Deux heures d'apprentissage guidé	10
Activité Colab — 30 minutes	11
Bilan solo — 30 minutes	11
Transfert et critères observables	11
Module 2 — Organisation Astro	12
Fichiers à examiner	12
Deux heures d'apprentissage guidé	12
Activité Colab — 30 minutes	13
Bilan solo — 30 minutes	13
Transfert et critères observables	13
Module 3 — Scène Three.js et coordonnées	14
Code et matériel	14
Deux heures d'apprentissage guidé	14
Activité Colab — 30 minutes	15
Bilan solo — 30 minutes	15
Transfert et critères observables	15
Quand deux interactions partagent le même geste	16
Module 4 — Vitesse et inertie	17
Du geste au mouvement	17
Deux heures d'apprentissage guidé	17
Activité Colab — 30 minutes	18
Bilan solo — 30 minutes	18

Transfert et critères observables	18
Commandes de l'aperçu du Module 4	19
Module 5 — Interpolation et élasticité	20
Intuition et code	20
Deux heures d'apprentissage guidé	20
Activité Colab — 30 minutes	21
Bilan solo — 30 minutes	21
Transfert et critères observables	21
Module 6 — Particules et ressources	22
Ce que contient une particule	22
Deux heures d'apprentissage guidé	22
Activité Colab — 30 minutes	23
Bilan solo — 30 minutes	23
Transfert et critères observables	23
Module 7 — Parcours, états et mémoire	24
Code du parcours	24
Deux heures d'apprentissage guidé	24
Activité Colab — 30 minutes	25
Bilan solo — 30 minutes	25
Transfert et critères observables	25
Commandes de l'aperçu du Module 7	25
Module 8 — Assistant, preuves et capacités	27
Code, trace et appel réel	27
Deux heures d'apprentissage guidé	27
Activité Colab — 30 minutes	28
Bilan solo — 30 minutes	28
Relier les productions à la fin de la leçon	28
Transfert et critères observables	29
Projet personnel — adapter, intégrer et expliquer	30
Calendrier — seize jours, sans double comptage	30
Cadrage du projet — 1 heure avec l'enseignant	30
Travail autonome du projet — 6 heures	31
Remise sélectionnée	31
Critères observables — une grille de revue	32
Stockage et permissions	32
Les neuf supports web élève	34
Qualification technique et lecture des preuves	35
Résultats courants et limites	35
Lire le corpus public admis	36
Les huit archives sélectionnées au Module 8	37
Documents et critères encore ouverts	37
Chronologie historique — résultats conservés	37
Historique — Chronologie des qualifications conservées	37
Historique — Module 4 — correctif et retest observé	45
Historique — Module 7 — correctif et retest observé	45
Historique — Modules 3 et 8 — intégration du geste et retest observé	45
Historique — Retour d'usage et prochaine revue	46

Historique — Lire la suspension du corpus public	46
Partie II — accompagner et enseigner	47
Préparer une séance	47
Examiner sans confondre les preuves	47
Intervenir à partir d'une difficulté	48
Choisir l'aide	48
Préparer l'accessibilité	48
Fiche enseignant 1 — Geste, état et rendu	49
Une activité avec l'élève et son assistant	49
Déroulé et script de séance	49
Script enseignant — séquence de 5 minutes	49
Repérer et travailler une difficulté	50
Transfert et retour humain	50
Fiche enseignant 2 — Organisation Astro	51
Une activité avec l'élève et son assistant	51
Déroulé et script de séance	51
Script enseignant — séquence de 5 minutes	51
Repérer et travailler une difficulté	52
Transfert et retour humain	52
Fiche enseignant 3 — Scène Three.js et coordonnées	53
Une activité avec l'élève et son assistant	53
Déroulé et script de séance	53
Script enseignant — séquence de 5 minutes	53
Repérer et travailler une difficulté	54
Transfert et retour humain	54
Fiche enseignant 4 — Vitesse et inertie	55
Une activité avec l'élève et son assistant	55
Déroulé et script de séance	55
Script enseignant — séquence de 5 minutes	55
Repérer et travailler une difficulté	56
Transfert et retour humain	56
Fiche enseignant 5 — Interpolation et élasticité	57
Une activité avec l'élève et son assistant	57
Déroulé et script de séance	57
Script enseignant — séquence de 5 minutes	57
Repérer et travailler une difficulté	58
Transfert et retour humain	58
Fiche enseignant 6 — Particules et ressources	59
Une activité avec l'élève et son assistant	59
Déroulé et script de séance	59
Script enseignant — séquence de 5 minutes	59
Repérer et travailler une difficulté	60
Transfert et retour humain	60
Fiche enseignant 7 — Parcours, états et mémoire	61
Une activité avec l'élève et son assistant	61
Déroulé et script de séance	61

Script enseignant — séquence de 5 minutes	61
Repérer et travailler une difficulté	62
Transfert et retour humain	62
Fiche enseignant 8 — Assistant, preuves et capacités	63
Une activité avec l'élève et son assistant	63
Déroulé et script de séance	63
La découverte de fin de leçon	63
Script enseignant — séquence de 5 minutes	64
Repérer et travailler une difficulté	65
Transfert et retour humain	65
Fiche enseignant projet — cadrage et clôture	66
Une heure de cadrage	66
Six heures solo, suivies de la clôture	66
Une heure de clôture	66
Script enseignant — séquence de 5 minutes	67
Revue de projet	67
Conclusion générale — une production que l'on peut expliquer	68
Annexes	69
A — Remettre et reprendre un travail	69
B — Les vingt-cinq outils dans le contexte pédagogique	69
C — Installer le plugin dans son Studio	70
D — Carte du code	71
E — Glossaire / Glossary	71
F — Symboles et unités	72
G — Index des notions / Concept Index	73
H — Sources et références	73
Mot de clôture	73
Empreintes intégrales de lecture	74

Orbit Formation

Construire, vérifier et enseigner avec son assistant personnel

Jean-Sébastien Beaulieu · Édition pédagogique 1.0 · Français

Ce manuel accompagne une formation de **40 heures**. Sa première partie aide à comprendre et pratiquer les mécanismes. Sa seconde partie aide l'enseignant à les faire observer, expliquer et transférer. Les fichiers du cours, les interventions de l'assistant et les modifications de l'élève gardent une attribution distincte.

Note d'édition et de responsabilité

Le code et les documents possèdent des versions différentes : `orbit-course-1.0.0` identifie le cours ; la version d'un landing ou d'un dossier désigne son propre objet. Ce document est une source préparée pour revue. La disponibilité d'un PDF, d'une page Canva ou d'un outil n'est pas un résultat d'apprentissage. Les conditions techniques sont réunies dans la section de qualification et dans le reçu maintenu.

Les droits des ressources citées restent ceux de leurs auteurs et de leurs licences. Cette édition identifie l'intention pédagogique et ses sources ; elle n'attribue aucune nouvelle licence aux documents externes. Le partage des travaux d'un élève suit son choix et les droits de son environnement.

Collaboration — déclaration à examiner lors de la revue d'auteur

J'ai utilisé Codex à son plein potentiel comme partenaire de recherche — pour la cartographie du code, la comparaison des sources, l'organisation des preuves, les contrôles de cohérence, le contrôle éditorial et la préparation des livrables. J'ai formulé l'intention, défini le périmètre, interprété les résultats, arbitré les conclusions et conservé chaque décision publique. Cette collaboration élargit ma capacité d'investigation ; le jugement, la responsabilité, la qualité d'auteur et la signature finale restent sous mon autorité.

À qui s'adresse le manuel ?

À un élève qui apprend avec son assistant personnel, et à l'enseignant qui accompagne ses essais. Les notions sont introduites depuis une action visible et un fichier réel. Un exemple complet peut être demandé à l'assistant : l'élève vérifie ensuite ce qu'il conserve, l'explique ou l'applique à une autre situation.

Comment l'utiliser

Lire la fiche du module, retrouver ses fichiers et poser une prédiction. Exécuter l'activité Colab avec une modification contrôlée. Choisir un rendu à remettre. Pendant la séance, expliquer une

observation et essayer une situation nouvelle. La seconde partie du manuel fournit les scripts et les pistes de revue à l'enseignant ; les documents web élève se distribuent au rythme du cours.

Les mots **fourni, modifié, déclaré exécuté, vérifié techniquement et examiné humainement** désignent des étapes différentes. Une aide peut être expliquée franchement. Une question ouverte et une limite donnent à l'enseignant un point de départ concret.

Table des matières

1. Le parcours de 40 heures et les destinations du travail.
2. Partie I — apprendre : huit modules et projet personnel.
3. Partie II — enseigner : huit fiches de séance et projet.
4. Annexes : capacités, remise, code, glossaire, symboles et sources.

La table des matières paginée sera construite lors de l'export DOCX/PDF.

Le parcours de 40 heures

Partie	Avec l'enseignant	Solo avec l'assistant personnel	Total
Huit modules	8 × 1 h	8 × 3 h	32 h
Projet personnel	2 h	6 h	8 h
Formation complète	10 h	30 h	40 h

Chaque période solo comprend **120 minutes de lecture et d'apprentissage guidé, 30 minutes d'activité Colab, puis 30 minutes de bilan**. Le webinaire ajoute une heure déjà comptée dans le module : 10 minutes de revue, 15 de clarification, 25 de construction et 10 de transfert.

Le projet suit **1 h de cadrage accompagné → 6 h solo → 1 h de clôture accompagnée**. Ses heures sont distinctes des modules. Dans le calendrier de seize jours, les six heures solo se distribuent comme 1 + 2 + 2 + 1 pendant les quatre derniers modules. La clôture suit les six heures solo.

Le rôle de chacun

Personne ou système	Travail et décision
Élève	Choisir son intention, ses essais, le niveau d'aide et les éléments à remettre ; expliquer sa décision
Assistant personnel	Expliquer, suggérer et comparer ; conserver les conditions, sources et questions ouvertes
Enseignant	Définir la progression, examiner le travail, choisir un exemple utile et enregistrer une revue réellement effectuée
Orbit	Fournir ressources, états, calculs déterministes, partage sélectionné et exports

Une réponse complète de l'assistant peut être utile. La question pédagogique devient : comment cette réponse est-elle vérifiée, expliquée et réutilisée ? Les critères portent sur des gestes observables et des raisonnements reliés au code.

Trois destinations à comprendre

Destination	Contenu et choix
Orbit en mémoire ou dans le profil navigateur	Session et fichiers sélectionnés ; la sauvegarde locale est un choix explicite, avec export avant fermeture si elle reste indisponible
Colab / Drive personnel	Copie du carnet, code, texte et sorties ; il s'agit du cloud Google et le partage peut inclure ces éléments
Sanity	Contenu accepté comme publiable, après examen du contenu et de la destination dans le plugin Studio

Le journal privé et la conversation complète ne sont pas des pièces obligatoires de remise. Partager avec l'assistant, déposer une proposition, choisir un moteur, remettre à l'enseignant et publier restent des décisions distinctes. Un stockage dans le profil navigateur n'est pas présenté comme un coffre chiffré.

Un notebook gratuit et une reprise explicite

Les exercices obligatoires utilisent un runtime CPU de Colab, aucun GPU, aucune API payante et aucune fonctionnalité d'IA intégrée obligatoire. Le Module 3 charge Three.js dans le navigateur depuis un CDN avec une alternative HTML. Les ressources Colab gratuites restent variables : conserver les exports et savoir reprendre fait partie du parcours.

Le code dans `student_files` passe à l'aperçu puis au ZIP. Le résultat JSON conserve prédiction, paramètres, observations choisies, explication, aide reçue, limites et question ouverte. Le ZIP contient le code choisi, une copie rejouable du notebook, les instructions et un manifeste SHA-256. Son import conserve le statut `external-declared`.

La fiche de Module 2 distingue aperçu et compilation Astro. Celle du Module 8 distingue trace préparée et appel natif. Les vérifications sur Colab, Orbit, un projet assemblé et un second Studio conservent chacune leur environnement.

Partie I — comprendre, expérimenter et construire

Le parcours commun est **observer** → **questionner** → **expliquer** → **prédire** → **expérimenter** → **conserver une preuve** → **transférer**. Ouvrir le fichier indiqué avant de modifier sa valeur. L'enseignant pourra examiner un texte, un schéma ou du code annoté ; une forme unique de réponse n'est pas imposée.

Module 1 — Geste, état et rendu

Objectif : expliquer quelle donnée un événement modifie et comment le rendu en dépend.

À la fin de l'activité, vous devez pouvoir retrouver la position, le pointeur actif et la phase du geste dans le code, puis expliquer un relâchement et une interruption. La fiche fournit une méthode ; votre compréhension sera examinée pendant le webinaire.

Ce module occupe les jours 1–2 : **3 h solo, puis 1 h avec l'enseignant**. Le notebook et le modèle sont fournis. Vos modifications et votre explication restent attribuées séparément à vous et à votre assistant.

Avant de commencer

Ouvrez le notebook élève dans Colab, puis conservez une copie personnelle. Un compte Google gratuit et un runtime CPU suffisent. L'assistant personnel peut fournir un exemple complet ; vous choisissez ce que vous essayez et remettez.

Le code d'étude est `interaction-state.js`. La copie de correction est conservée dans l'espace enseignant. La mission du catalogue est définie dans `catalog.ts`.

Deux heures d'apprentissage guidé

Temps	Votre travail avec l'assistant	Trace sélectionnée
0–30 min	Lire le modèle. Repérer <code>state.position</code> , phase et <code>pointerId</code> . Dessiner « événement → donnée → rendu ».	Un petit schéma et une question
30–60 min	Lire <code>pointerdown</code> et <code>pointermove</code> . Expliquer pourquoi un survol seul ne déplace pas l'élément.	Trois phrases reliées aux lignes concernées
60–90 min	Comparer <code>pointerup</code> , <code>pointercancel</code> et <code>lostpointercapture</code> . Lire les ressources officielles.	Une différence entre relâchement et annulation
90–120 min	Reprendre le parcours au clavier. Préparer une prédiction avec deux pressions vers la droite.	Valeur initiale, valeur testée, conditions

Demande possible à l'assistant : « Explique-moi `pointerId` à partir de ce fichier. Donne un

scénario normal et un scénario interrompu, puis demande-moi quelle donnée doit être restaurée.
» Vous pouvez demander la solution ; votre propre trace décrit ensuite ce que vous avez vérifié.

Dans ce modèle, x et y sont normalisés entre 0 et 1. $x = 0.5$ signifie le milieu de la zone, pas 0,5 pixel. L'état est copié avant transmission au rendu, afin de garder une séparation explicite entre les données et leur présentation.

```
export const interactionOptions = { keyboardStep: 0.04 };
```

Cette valeur règle le pas clavier. Elle ne change ni la largeur de la zone ni la vitesse de la souris. Sous capture du pointeur, sortir de la zone ne signifie pas automatiquement annuler le geste.

Activité Colab — 30 minutes

1. **Prédire — 5 min.** À partir de $x = 0.5$, annoncer la position attendue après deux flèches droites avec un pas de 0,04, puis 0,08. Ajouter une hypothèse sur le pointeur actif après annulation. Écrire la prédiction avant l'aperçu.
2. **Modifier et observer — 15 min.** Dans `student_files`, changer uniquement `keyboardStep: 0.04` en `keyboardStep: 0.08`. Exécuter l'aperçu depuis cette même variable. Comparer deux pressions ; essayer un geste puis son relâchement. Repérer dans le code la transition d'annulation. Une annulation native dépend du navigateur et du périphérique ; ne la déclarez pas observée si vous avez seulement lu le code ou pressé Échap.
3. **Expliquer et exporter — 10 min.** Noter le résultat observé, la variable conservée et une limite. Remplir l'aide reçue et l'explication, puis exporter le ZIP. Le ZIP contient le code effectivement présent dans `student_files`, pas un corrigé substitué.

Piste de vérification, à consulter après votre prédiction : deux pressions ajoutent 0,08 avec le premier pas et 0,16 avec le second, tant que la borne droite n'est pas atteinte. Le code place `pointerId` à `null` à la fin d'un geste. Ce calcul attendu ne remplace pas votre essai.

Bilan solo — 30 minutes

Consacrer 10 minutes au choix des traces à remettre, 10 minutes à une explication personnelle et 10 minutes à la question pour le webinaire. Conserver le ZIP, le scénario utilisé et les éventuels écarts. Une capture seule montre une image ; l'explication doit nommer l'événement et l'état.

Votre carnet personnel et ses sorties sont dans Colab/Drive. Vous choisissez le rendu transmis à l'enseignant. Aucune conversation privée complète n'est requise.

Transfert et critères observables

Créer une carte déplaçable qui dispose aussi d'une commande clavier. Vous devez pouvoir : nommer la donnée modifiée ; distinguer relâchement, sortie de zone et annulation ; montrer la remise à zéro du pointeur actif ; sélectionner le même élément sans souris. Un scénario vérifié et une reformulation valent davantage qu'une définition copiée.

Les tests logiciels examinent des comportements du modèle. Ils ne certifient ni un geste tactile réel ni votre compréhension.

Ressources primaires et limites : Capture du pointeur — MDN · Événement `pointercancel` — MDN · Projet et règles de remise

Module 2 — Organisation Astro

Objectif : séparer contenu, présentation et comportement. Vous apprendrez à retrouver le fichier responsable d'une modification et à réutiliser une carte avec un autre contenu. Le module occupe les jours 3–4 : **3 h solo, puis 1 h avec l'enseignant.**

Le prérequis est d'avoir rencontré le parcours événement → état → rendu du Module 1. Votre assistant peut expliquer directement Astro ; votre travail consiste ensuite à vérifier cette explication dans les fichiers et à préparer une modification précise.

Fichiers à examiner

- ExplorationCard.astro fournit le titre, la section, la liste et le paragraphe de description.
- exploration-card.js crée les boutons, associe un identifiant à chaque élément et actualise la description.
- Notebook élève ; la copie de correction conservée par l'enseignant.

La carte Astro déclare deux propriétés : `id` et `title`. Sa liste HTML est vide au départ ; le contrôleur fourni y ajoute les éléments. Cette séparation permet de changer les données sans recopier les événements.

```
export const cardOptions = { descriptionPrefix: 'Mon observation : ' };
```

Le préfixe appartient à la description affichée par le contrôleur. Modifier ce texte ne doit pas ajouter un bouton. Dans le fichier Astro, `aria-label` reprend le titre de la section : un libellé reste nécessaire même si la carte change d'apparence.

Deux heures d'apprentissage guidé

Temps	Travail avec l'assistant	Trace sélectionnée
0–30 min	Lire le composant Astro. Retrouver propriétés, section, titre, liste et description.	Arborescence courte et rôle de chaque élément
30–60 min	Lire <code>createCards</code> , <code>select</code> et <code>dispose</code> . Suivre un identifiant depuis les données jusqu'au bouton.	Un parcours de sélection annoté
60–90 min	Lire les composants Astro et les îlots. Distinguer page construite, HTML reçu et comportement navigateur.	Explication de trois responsabilités

Temps	Travail avec l'assistant	Trace sélectionnée
90–120 min	Choisir un autre contenu pour une carte et préparer une modification unique.	Prédiction et contenu personnel d'essai

Demande possible : « Voici ces deux fichiers. Explique qui fournit le HTML, qui remplit la liste et qui réagit à la sélection. Donne un exemple complet, puis demande-moi où je changerais le titre sans changer l'état du geste. » Votre assistant aide à lire ; les chemins et symboles servent à contrôler sa réponse.

Activité Colab — 30 minutes

1. **Prédire — 5 min.** Écrire ce qui changera si `descriptionPrefix` devient `Ma décision expliquée` :. Prédire aussi ce qui restera identique : nombre de boutons, identifiants et sélection.
2. **Modifier et observer — 15 min.** Dans `student_files`, changer seulement le préfixe de `exploration-card.js`. Exécuter l'aperçu, sélectionner deux éléments et comparer la description. Retrouver ensuite le titre dans `ExplorationCard.astro`, sans attribuer la modification au mauvais fichier.
3. **Expliquer et exporter — 10 min.** Décrire la responsabilité de chaque fichier ; conserver votre aide reçue, une limite et une question. Exporter les deux fichiers dans le même ZIP.

L'aperçu du notebook présente une contrepartie HTML préparée. **Il ne compile pas le composant Astro.** L'exercice permet de comparer le comportement de la carte ; la compilation du vrai composant appartient au parcours de projet vérifié séparément.

Piste après tentative : `createCards` utilise `textContent` pour les titres et descriptions, puis `aria-pressed` pour la sélection. Rechercher `select(id)` permet de relier le bouton aux données. Une amélioration de style se vérifie ensuite dans le projet, avec le titre et le clavier préservés.

Bilan solo — 30 minutes

Prendre 10 minutes pour choisir votre modification et les traces, 10 minutes pour expliquer la séparation des responsabilités et 10 minutes pour préparer le webinaire. Votre rendu comprend le ZIP, un exemple de carte pour votre sujet et une phrase qui distingue aperçu HTML et compilation Astro.

Une suggestion de l'assistant, un code exécuté et une explication comprise ont des statuts distincts. Vous pouvez signaler « je retrouve le fichier, mais je ne peux pas encore expliquer ce contrôleur » : cette limite prépare une revue utile.

Transfert et critères observables

Préparer une carte pour un thème personnel. L'élève retrouve le rôle de chaque fichier, change une propriété sans dupliquer la logique et explique la différence entre l'aperçu et la compilation. La sélection doit garder un libellé et un état accessibles.

Ressources et statut : Composants Astro — documentation officielle · Architecture en îlots — documentation officielle · Mission et critères du catalogue · Règles du projet et de la remise

Module 3 — Scène Three.js et coordonnées

Objectif : relier objet, caméra, coordonnées et sélection. Vous allez observer comment la caméra modifie une projection, puis retrouver le même élément dans une représentation HTML. Le module occupe les jours 5–6 : **3 h solo, puis 1 h avec l’enseignant.**

Le prérequis est de distinguer structure et comportement dans une carte. La scène est une autre manière de présenter les éléments ; leur identifiant et leur contenu restent disponibles dans la liste HTML.

Code et matériel

Ouvrir le notebook élève et scene-controller.js. La copie de correction conservée par l’enseignant est séparée. Le notebook utilise un runtime CPU ; Three.js 0.181.2 s’exécute dans le navigateur depuis un CDN. Le refus du réseau ou l’absence de WebGL conserve le parcours HTML, sans valider une scène 3D.

```
export const sceneOptions = { cameraDistance: 5 };
```

Le contrôleur reçoit THREE depuis son hôte. Il prépare scène, caméra, objets et lumière. Il ne crée pas sa propre boucle de rendu : l’hôte décide quand appeler `render()`. Le même `item.id` relie sphère, sélection et fiche HTML.

Les données de position de l’activité sont normalisées. Le contrôleur les transforme vers la scène ; la caméra projette ensuite la scène vers l’écran. Déplacer une caméra n’est donc pas déplacer les données de l’objet.

Deux heures d’apprentissage guidé

Temps	Travail avec l’assistant	Trace sélectionnée
0–30 min	Lire scène, caméra, matériau et lumière. Associer chaque mot à un objet du code.	Schéma de la présentation
30–60 min	Suivre un identifiant de la liste HTML à une sphère. Repérer le raycasting.	Chemin « clic → identifiant → fiche »
60–90 min	Lire la projection et le redimensionnement. Distinguer coordonnées normalisées, scène et écran.	Une explication avec trois coordonnées différentes

Temps	Travail avec l'assistant	Trace sélectionnée
90–120 min	Préparer la comparaison caméra 5/7 avec positions conservées. Examiner le parcours sans WebGL.	Prédiction et conditions de l'essai

Demande possible : « Donne-moi une analogie de la caméra, puis relie-la à `camera.position.z` et à `item.position`. Demande-moi quelle valeur change si l'objet paraît plus petit. » L'analogie aide à démarrer ; l'explication finale revient aux données du code.

Activité Colab — 30 minutes

1. **Prédire — 5 min.** Avec les mêmes objets, annoncer l'effet attendu d'une caméra passant de 5 à 7. Préciser si votre hypothèse concerne la taille affichée ou la position enregistrée.
2. **Modifier et observer — 15 min.** Dans `student_files`, remplacer `cameraDistance: 5` par `cameraDistance: 7`. Exécuter l'aperçu. Choisir une sphère puis le bouton HTML correspondant ; comparer l'identifiant sélectionné. Observer un changement de taille de la zone si l'interface le permet. Si la scène ne démarre pas, consigner le refus et utiliser la liste, sans inventer une observation 3D.
3. **Expliquer et exporter — 10 min.** Décrire ce qui a changé et ce qui reste dans les données. Noter navigateur, disponibilité WebGL, aide reçue et limite. Exporter le contrôleur réellement modifié.

Piste après tentative : reculer la caméra modifie la projection, avec les positions d'objet inchangées. Une sphère et un bouton HTML peuvent désigner le même élément. Un identifiant commun ne suffit toutefois pas à prouver que tous les gestes tactiles ou tous les redimensionnements fonctionnent.

Bilan solo — 30 minutes

Choisir les traces pendant 10 minutes, rédiger votre explication pendant 10 minutes et préparer une question pendant 10 minutes. Votre rendu nomme l'élément choisi, les deux distances, la disponibilité de la scène et une limite.

La position, la couleur ou la taille d'un objet **ne mesure pas la vérité d'une affirmation**. La scène fournit ici une représentation artistique et pédagogique ; elle ne constitue pas une simulation scientifique validée.

Transfert et critères observables

Créer trois éléments consultables à la fois dans une liste et une scène. L'élève distingue coordonnées d'écran et de scène, montre le lien d'une sélection à sa fiche et retrouve le contenu sans WebGL. Il explique le rôle d'un redimensionnement et les ressources à nettoyer à la fermeture.

Le contrôleur possède `dispose()` pour libérer événements, géométries, matériaux et rendre. La présence de cette fonction dans le code demande encore une vérification de son appel dans le parcours complet.

Ressources et statut : Fondamentaux Three.js — manuel officiel · Raycaster — API Three.js · Mission du catalogue · Projet et règles de remise

Quand deux interactions partagent le même geste

Dans `scene-controller.js`, `pointerTarget` indique où écouter le geste. Sa valeur par défaut est le canvas de `Three.js`. Le calcul de projection utilise toujours le rectangle du canvas : la cible d'événement et la surface dessinée ont des responsabilités différentes.

Le Module 1 peut capturer le pointeur sur le stage. Dans cet assemblage, le relâchement est alors envoyé au stage. Un écouteur placé seulement sur le canvas risque de ne jamais le recevoir, même si la scène se dessine correctement. L'hôte fourni dans `student-frontend.js` raccorde explicitement les deux briques :

```
scene = createSceneController({
  container: root.querySelector('[data-three]'),
  THREE, items, onSelect: select, pointerTarget: stage
});
```

Ce code est fourni par le cours. Il ne remplace pas le fichier exporté de l'élève. Le début du geste doit provenir du canvas ou de son chemin d'événements ; un déplacement supérieur à six pixels est considéré comme un glisser et ne sélectionne pas une sphère. Une annulation efface le geste en attente. Le nettoyage retire les écouteurs, l'observateur de taille et les ressources de rendu.

Petit transfert : dessiner deux rectangles, le stage et son canvas. Indiquer où le pointeur commence, quelle surface le capture, où arrive son relâchement et quel rectangle transforme ses coordonnées. Puis expliquer ce que l'on doit conserver lorsqu'une caméra recule. Les essais réellement observés, les incidents et le retest de l'assemblage gardent leurs reçus dans la qualification ; cet exemple de code seul n'en démontre pas le comportement.

Module 4 — Vitesse et inertie

Objectif : comprendre le mouvement après relâchement. Vous allez définir position, vitesse et durée, puis comparer deux amortissements. Le module occupe les jours 7–8 : **3 h solo, puis 1 h avec l'enseignant.**

Le geste, ses phases et les coordonnées ont été rencontrés. La nouvelle question est : quelle donnée permet à un élément de continuer son mouvement lorsque le pointeur n'est plus actif ?

Du geste au mouvement

Le modèle inertia.js expose grab, move, release, cancel, step et snapshot. Ouvrir le notebook élève, avec la copie de correction conservée par l'enseignant.

```
export const inertiaOptions = { damping: 0.65, restitution: 0.9 };
```

La position est normalisée entre 0 et 1 ; la vitesse utilise ces unités par seconde ; dt est une durée en secondes. À vitesse constante de 0,2 unité/seconde pendant 0,05 seconde, le déplacement serait 0,01 unité. Ce petit calcul sert à comprendre les unités avant l'amortissement.

Le code utilise $\text{decay} = \text{Math.exp}(-\text{damping} * \text{dt})$. Si k désigne l'amortissement et t une durée, la vitesse devient $v(t) = v(0) \times \exp(-k \times t)$ entre les rebonds. Pour $k > 0$, le déplacement correspondant est $v(0) \times (1 - \exp(-k \times t)) / k$. Le code traite aussi $k = 0$. Ce modèle déterministe est un effet d'interface, sans validation physique annoncée.

Deux heures d'apprentissage guidé

Temps	Travail avec l'assistant	Trace sélectionnée
0–30 min	Associer x/y, vx/vy et dt à leurs unités. Faire un petit déplacement à vitesse constante.	Calcul court et unités
30–60 min	Lire comment deux positions et deux instants donnent une vitesse pendant le geste. Comparer release et cancel.	Une séquence de données
60–90 min	Lire l'amortissement, le plafond de dt et les rebonds. Expliquer pourquoi le survol ne réagrippe pas.	Une limite et un scénario

Temps	Travail avec l'assistant	Trace sélectionnée
90–120 min	Préparer la comparaison de 0,65 et 1,3 avec restitution conservée à 0,9.	Prédiction et conditions

Demande possible : « Pars d'une position et d'une vitesse simples. Explique pourquoi on multiplie par une durée en secondes. Fais-moi vérifier l'unité de chaque valeur avant de comparer les amortissements. » Demander ensuite à l'assistant de relier son exemple à step, sans créer un second moteur parallèle.

Activité Colab — 30 minutes

1. **Prédire — 5 min.** Annoncer quel amortissement devrait raccourcir la trajectoire après relâchement. Conserver la zone, la restitution et un geste aussi comparable que possible.
2. **Modifier et observer — 15 min.** Dans `student_files`, remplacer uniquement `damping: 0.65` par `damping: 1.3`. Exécuter l'aperçu et comparer deux lancers. Croiser l'objet avec le pointeur, sans cliquer. Tester aussi un arrêt prolongé avant relâchement : dans ce modèle, plus de 0,12 seconde sans nouvel échantillon annule l'élan estimé. Consigner ces gestes séparément.
3. **Expliquer et exporter — 10 min.** Décrire la trajectoire, la variable changée et la comparabilité imparfaite des gestes. Remplir aide reçue, explication et limites, puis exporter le code modifié.

Piste après tentative : un amortissement supérieur réduit plus rapidement la vitesse. Deux lancers humains ne garantissent pas une vitesse initiale identique. Pour isoler exactement l'effet du moteur, les tests déterministes utilisent des entrées contrôlées ; votre observation d'interface indique ce que vous avez effectivement essayé.

Bilan solo — 30 minutes

Sélectionner les essais pendant 10 minutes, expliquer l'effet pendant 10 minutes et préparer une question pendant 10 minutes. Votre bilan doit distinguer observation visuelle, calcul prévu et test déterministe. Une cadence de rendu variable peut affecter l'expérience ; elle ne transforme pas un écart en preuve de la théorie physique.

Conserver les paramètres et les unités est indispensable pour rejouer un résultat. Une vidéo sans paramètres reste une trace partielle.

Transfert et critères observables

Adapter l'inertie à une carte HTML. L'élève définit les unités, conserve des conditions comparables, montre pourquoi le pointeur passif ne change pas la vitesse et décrit ce qui se fige en mode statique. Le modèle garde la position courante et sa vitesse pendant le gel ; le temps suspendu n'est pas à rattraper.

Les tests de briques comparent notamment des durées contrôlées et des états figés. Leur résultat courant appartient au bilan de validation.

Ressources et statut : `requestAnimationFrame` — MDN · Mission du catalogue · Règles du projet et de la remise

Commandes de l'aperçu du Module 4

Le notebook corrigé conserve deux gestes distincts : les flèches déplacent par pas de 0,04 et gardent v_x/v_y à zéro pendant le geste; leur relâchement termine le geste sans lancer. Le glisser-relâcher au pointeur estime une vitesse et produit l'inertie. Espace fige/reprend l'image et le temps; Échap annule un geste actif. En vol libre, le gel conserve position et vitesse, et sa durée n'est pas rattrapée. Un geste actif est terminé sans lancement lors du gel.

Ces consignes suivent l'aperçu préparé dans le notebook du Module 4. Le reçu antérieur qui avait détecté l'absence du clavier reste historique; la qualification du correctif se lit dans la section commune des preuves.

Module 5 — Interpolation et élasticité

Objectif : distinguer une transition vers une cible d'un mouvement oscillant. Vous allez relier cible, position, vitesse, ressort et amortissement, puis expliquer un dépassement. Le module occupe les jours 9–10 : **3 h solo, puis 1 h avec l'enseignant.** Le projet personnel commence aussi pendant ces jours avec des heures distinctes : consulter PROJECT.md.

Le prérequis est d'avoir rencontré vitesse et durée. Le modèle de ce module rend une transition d'interface réglable ; il ne revendique pas une simulation physique validée.

Intuition et code

L'activité Colab utilise transitions.js. La copie de correction conservée par l'enseignant reste séparée.

Une interpolation rapproche une valeur de sa cible selon une règle. Un ressort conserve aussi une vitesse : même en arrivant à la cible, il peut continuer et la dépasser. Une animation peut paraître douce avec plusieurs mécanismes ; son apparence ne suffit pas à identifier le code.

```
export const transitionOptions = { stiffness: 36, damping: 12 };
```

Ici, `value` est une échelle sans unité, `target` sa cible et `velocity` la variation de cette échelle par seconde. Le terme `stiffness × (target - value)` attire vers la cible ; `damping × velocity` freine. `step` borne la durée et la découpe pour l'intégration. Les bornes de ce modèle sont explicites ; des paramètres arbitraires ne deviennent pas sûrs parce qu'une seule animation semble fonctionner.

Deux heures d'apprentissage guidé

Temps	Travail avec l'assistant	Trace sélectionnée
0–30 min	Dessiner une approche directe et un mouvement qui dépasse. Nommer valeur et cible.	Deux schémas et une différence
30–60 min	Lire <code>setTarget</code> , <code>velocity</code> et <code>step</code> . Suivre un calcul d'accélération avec des nombres simples.	Une ligne de calcul commentée
60–90 min	Lire amortissement, sous-pas et mode statique. Distinguer figer la valeur et la remettre au départ.	Un scénario de gel/reprise

Temps	Travail avec l'assistant	Trace sélectionnée
90–120 min	Préparer la comparaison de 12 et 5 avec raideur 36 et cible conservées.	Prédiction, variable et conditions

Demande possible : « Explique un ressort à partir de `target` - `value` et `velocity`, puis montre ce qui manque dans une simple interpolation. Demande-moi de prédire un dépassement avant de l'essayer. » Une solution complète est permise ; votre explication revient ensuite au code et à votre observation.

Activité Colab — 30 minutes

1. **Prédire — 5 min.** Annoncer l'effet attendu du passage de `damping`: 12 à `damping`: 5, avec la même cible et `stiffness`: 36. Prédire aussi ce que signifie activer le mode figé pendant le mouvement.
2. **Modifier et observer — 15 min.** Modifier uniquement l'amortissement dans `student_files`, puis exécuter l'aperçu. Comparer dépassement et stabilisation. Activer le gel à un instant choisi et examiner la valeur courante. La reprendre sans demander au modèle de rattraper le temps suspendu. Restaurer la valeur initiale pour un dernier essai.
3. **Expliquer et exporter — 10 min.** Écrire ce que vous avez effectivement vu, votre aide reçue et une limite. Exporter la version choisie ; préciser si vous avez conservé ou restauré le paramètre.

Piste après tentative : une diminution de l'amortissement permet davantage d'oscillation dans cet essai. `step(dt, true)` conserve la valeur et la vitesse présentes. Cela ne signifie pas rendre le mouvement invisible ; l'image actuelle reste consultable.

Bilan solo — 30 minutes

Passer 10 minutes à sélectionner les résultats, 10 minutes à expliquer un dépassement et 10 minutes à préparer une question pour la revue. Distinguer « j'ai observé cette oscillation » de « j'ai compris toutes les conditions de stabilité ». Une incertitude conservée aide l'enseignant à choisir l'exemple suivant.

Le projet personnel avance en parallèle ; ses heures ne sont pas prises sur ce bilan de module.

Transfert et critères observables

Appliquer la transition à l'ouverture d'une fiche. L'élève nomme le paramètre changé, distingue dépassement et stabilisation, conserve un mode figé utilisable et montre où `dispose()` arrête la ressource. Il explique la différence entre valeur cible et valeur présente dans un nouvel exemple.

Les tests de briques examinent les bornes et certains états statiques. Un test réussi ne dispense pas d'observer le geste et de reformuler le mécanisme.

Ressources et statut : Préférence de mouvement réduit — MDN · Mission et critères du catalogue · Projet personnel

Module 6 — Particules et ressources

Objectif : distinguer population, durée de vie, calcul et rendu. Vous allez modifier une population bornée, comparer les effets et expliquer les ressources à libérer. Le module occupe les jours 11–12 : **3 h solo, puis 1 h avec l'enseignant, avec 2 h solo de projet supplémentaires et distinctes.**

Le prérequis est de lire une position, une vitesse et une durée. Une particule de cet exercice est un petit ensemble de données ; elle n'est pas une particule physique mesurée.

Ce que contient une particule

Ouvrir `particle-layer.js`, le notebook élève et, pour l'enseignant seulement, la copie de correction conservée par l'enseignant.

```
export const particleOptions = { count: 64, lifetime: 3, seed: 17 };
```

Chaque élément contient position x/y , vitesse v_x/v_y et âge. La population est créée une fois. Un élément arrivé en fin de vie réutilise son emplacement avec un âge nul et une nouvelle position. La limite du modèle est de 256 éléments ; la durée de vie est bornée entre 0,2 et 10 secondes.

La graine `seed` rend la suite de nombres simulés répétable pour les mêmes conditions. Elle ne prouve ni la validité physique du modèle ni une performance identique sur tous les appareils. La fonction `dispose()` vide les données de la couche ; le contrôleur de scène libère séparément ses ressources graphiques.

Deux heures d'apprentissage guidé

Temps	Travail avec l'assistant	Trace sélectionnée
0–30 min	Lire les données d'une particule. Distinguer nombre, âge et durée de vie.	Une particule annotée
30–60 min	Lire création et réutilisation. Expliquer pourquoi la population n'augmente pas à chaque image.	Un parcours de fin de vie
60–90 min	Distinguer calcul JavaScript, dessin du navigateur, GPU et runtime Python. Lire le nettoyage <code>Three.js</code> .	Une mesure pertinente et une mesure hors portée

Temps	Travail avec l'assistant	Trace sélectionnée
90–120 min	Préparer l'essai 64/128 avec durée 3 et graine 17 conservées. Choisir le scénario et l'appareil à décrire.	Prédiction et conditions

Demande possible : « Suis une particule avant et après sa fin de vie. Explique ce qui change et ce qui reste constant. Aide-moi ensuite à choisir une mesure du navigateur, sans utiliser le temps Python comme mesure du rendu Three.js. »

Activité Colab — 30 minutes

1. **Prédire — 5 min.** Annoncer ce qui augmente lorsque count passe de 64 à 128. Prédire ce que vous pourriez voir et ce qu'un simple chronomètre du runtime ne permettrait pas de conclure.
2. **Modifier et observer — 15 min.** Dans `student_files`, changer seulement `count: 64` en `count: 128`. Garder `lifetime: 3` et `seed: 17`. Comparer la population et le comportement visuel. Activer le mode figé puis reprendre. Restaurer 64 après l'essai. Le notebook dessine un aperçu Canvas : il n'est pas une mesure de coût GPU Three.js.
3. **Expliquer et exporter — 10 min.** Noter navigateur, scénario, population et durée. Si une métrique manque, écrire « indisponible » plutôt que zéro. Conserver observation, aide reçue et limite, puis exporter la version restaurée.

Piste après tentative : deux fois plus d'éléments signifie plus de données à mettre à jour et à dessiner. Cela ne garantit pas une durée de frame exactement doublée : appareil, navigateur, autres travaux et stratégie de rendu comptent. Une mesure de performance doit décrire sa source et ses conditions.

Bilan solo — 30 minutes

Choisir les traces pendant 10 minutes, expliquer population et ressources pendant 10 minutes, puis préparer une question pendant 10 minutes. Le rendu peut inclure une mesure absente ou un appareil peu performant : ces limites sont utiles pour choisir un effet facultatif.

Vous devez pouvoir distinguer « 128 éléments dans les données » de « 128 objets graphiques indépendants ». Le `render` peut organiser les données autrement ; il faut lire son chemin de code avant de généraliser.

Transfert et critères observables

Créer une couche facultative, bornée et désactivable qui signale une sélection. L'élève distingue population et durée, indique la source d'une mesure, conserve un libellé sans effet et montre le nettoyage à la fermeture. Une population identique après plusieurs cycles ne suffit pas à prouver toute l'absence de fuite ; le parcours complet doit être observé.

Le code garde les données présentes en mode figé. La couche peut être désactivée explicitement pour une préférence d'accessibilité distincte.

Ressources et statut : Nettoyage Three.js — manuel officiel · Contrôleur de scène et ressources graphiques · Mission du catalogue · Projet et conditions de remise

Module 7 — Parcours, états et mémoire

Objectif : gérer les transitions sans état bloqué. Vous allez conserver une sélection, limiter un historique et expliquer la restauration d'une vue. Le module occupe les jours 13–14 : **3 h solo**, puis **1 h avec l'enseignant**, avec **2 h solo de projet distinctes**.

Le Module 1 gère un geste ; celui-ci gère le parcours. « Quel pointeur est actif ? » et « Quelle fiche est ouverte ? » appartiennent à des états différents, même lorsque le même clic intervient dans les deux.

Code du parcours

Ouvrir `interaction-flow.js`, le notebook élève et, pour l'enseignant, la copie de correction conservée par l'enseignant.

```
export const flowOptions = { historyLimit: 32 };
```

Le modèle conserve `view`, `selection`, `elapsed` et `static`. Les vues autorisées sont `mission`, `experience`, `proofs` et `summary`. La sélection conserve un identifiant borné. `navigate` ajoute une entrée ; si le budget est dépassé, la plus ancienne sort.

L'aperçu Colab reçoit `host: null`. Son historique est interne : il ne pilote pas le bouton Retour de la page Colab. Dans le projet Astro, l'hôte navigateur relie les paramètres `learningView` et `learningSelection` à l'URL et écoute `popstate`. Les deux parcours sont vérifiés séparément.

Deux heures d'apprentissage guidé

Temps	Travail avec l'assistant	Trace sélectionnée
0–30 min	Dessiner liste → fiche → retour avec une sélection précise. Nommer ce qui doit être conservé.	Trois états du parcours
30–60 min	Lire <code>navigate</code> et <code>snapshot</code> . Repérer validation, ajout et retrait d'une entrée.	Une trace de cinq changements
60–90 min	Lire URL, <code>popstate</code> et visibilité. Distinguer historique interne et navigation navigateur.	Une condition de restauration
90–120 min	Préparer l'essai des limites 32/4 et le gel de la chronologie.	Prédiction et scénario

Demande possible : « Aide-moi à reproduire un retour à une mauvaise sélection. Montre quelles données restaurer, puis demande-moi de tracer cinq changements avec un historique limité à quatre. » L'assistant propose le scénario ; vous conservez la séquence effectivement utilisée.

Activité Colab — 30 minutes

1. **Prédire — 5 min.** Écrire quelle entrée devrait sortir après cinq changements lorsque `historyLimit` vaut 4. Choisir une séquence précise, avec au moins deux éléments sélectionnés. Prédire l'effet du mode figé sur `elapsed`.
2. **Modifier et observer — 15 min.** Dans `student_files`, comparer `historyLimit: 32` puis `historyLimit: 4` avec la même séquence. Consulter les entrées conservées et la sélection courante. Activer le gel, attendre, reprendre et examiner l'horloge. Ne pas appeler cet essai une validation du bouton Retour navigateur.
3. **Expliquer et exporter — 10 min.** Décrire l'entrée retirée, l'état restauré et les conditions de chronologie. Noter aide, limite et question, puis exporter le code effectivement choisi.

Piste après tentative : l'entrée la plus ancienne sort lorsque la cinquième est ajoutée à un budget de quatre. Le budget concerne ici les entrées de diagnostic internes, pas une suppression arbitraire de l'historique global du navigateur. Le gel suspend `elapsed` et ne doit pas ajouter le temps d'attente au retour.

Bilan solo — 30 minutes

Prendre 10 minutes pour sélectionner la séquence, 10 minutes pour expliquer un retour et 10 minutes pour préparer le webinaire. Le rendu conserve état initial, événements et état final. « Cela semble fonctionner » devient une vérification utile lorsqu'on peut retrouver le scénario précis.

Une trace locale n'est pas automatiquement un journal à publier. L'élève choisit les éléments à remettre ; aucune conversation complète n'est nécessaire.

Transfert et critères observables

Créer un parcours fiche → liste qui restaure la sélection. L'élève conserve vue et identifiant, explique l'entrée retirée par la borne, distingue historique interne et historique navigateur et vérifie le gel sans rattrapage de temps. Après fermeture, les événements doivent être nettoyés, pas ajoutés à nouveau sans limite.

Les tests logiciels de briques examinent notamment l'état interne. Le parcours navigateur demande une observation dans l'environnement cible.

Ressources et statut : History API — MDN · Page Visibility API — MDN · Mission du catalogue · Projet et reprise

Commandes de l'aperçu du Module 7

Les boutons des trois éléments affichent `element-1`, `element-2` et `element-3`. Tab puis Entrée ou Espace permettent de les choisir; le pointeur déclenche le même choix. Le snapshot affiche `selection` et les boutons signalent la sélection courante. Changer ensuite `mission`, `experience`, `proofs` ou `summary` conserve cet identifiant.

Préparer cinq choix précis, avec au moins deux identifiants différents, puis répéter la même séquence après avoir changé `historyLimit` de 32 à 4. Les choix d'élément et les changements

de vue ajoutent chacun une entrée. Noter l'entrée retirée et la sélection finale. Cet historique reste celui de l'iframe; le bouton Retour du navigateur est examiné dans le projet Astro. Le statut du retest du notebook corrigé se lit dans la qualification commune.

Module 8 — Assistant, preuves et capacités

Objectif : comprendre ce qu'un agent peut lire, calculer et proposer. Vous allez examiner permissions, révisions et provenance, puis découvrir comment raccorder vos productions. Le module occupe les jours 15–16 : **3 h solo, puis 1 h avec l'enseignant**. La dernière heure solo du projet et son heure de clôture sont **supplémentaires et distinctes**.

Les états, le rendu et le parcours ont été rencontrés. Une capacité d'agent ajoute une question : quel contenu est effectivement partagé, pour quelle révision et avec quelle autorité ?

Code, trace et appel réel

Ouvrir `register-capabilities.js` et le notebook élève. La copie de correction est réservée à l'enseignant.

Le module prépare **un outil du projet élève**, `student_get_learning_snapshot`. Il ne remplace pas le registre des vingt-cinq outils d'Orbit. Son adaptateur reçoit `canRead`, `getRevision`, `getSnapshot` et l'API navigateur. Il contrôle la permission avant de lire et vérifie la révision. Les résultats distinguent notamment `CONSENT_REQUIRED`, `STALE_REVISION`, `UNAVAILABLE` et `READY`.

```
if (!canRead()) return { state: 'CONSENT_REQUIRED' };
```

La présence de `READY` dans une trace préparée n'est pas la preuve d'un appel réel. Un assistant ne peut pas fabriquer une approbation humaine en ajoutant un champ. Le texte d'une source, d'un export ou d'une proposition est une donnée à examiner, pas une instruction qui change les permissions.

Deux heures d'apprentissage guidé

Temps	Travail avec l'assistant	Trace sélectionnée
0–30 min	Lire la capacité bornée. Identifier contenu autorisé, permission et révision.	Une fiche de capacité
30–60 min	Examiner la trace préparée du notebook. Repérer une permission absente, une version périmée ou une autorité usurpée.	Objection reliée à un champ

Temps	Travail avec l'assistant	Trace sélectionnée
60–90 min	Lire les outils d'Orbit, passage, portée et proposition. Distinguer présence d'une citation et pertinence.	Un exemple de limite de conclusion
90–120 min	Préparer un besoin utile pour votre assistant et la comparaison d'une limite de volume.	Prédiction et périmètre choisi

Demande possible : « Explique les conditions de lecture dans ce code. Ensuite, examine cette trace comme une donnée d'exercice : quelles affirmations ne puis-je pas vérifier à partir d'elle ? Propose une capacité utile et bornée pour mon projet. » Votre assistant peut expliquer directement ; vous conservez la justification de la permission et du contenu partagé.

Activité Colab — 30 minutes

1. **Prédire — 5 min.** Écrire pourquoi une trace préparée READY ne certifie ni un appel natif ni une revue humaine. Prédire l'effet d'une limite de snapshot passant de 12 000 à 6 000 caractères de sérialisation.
2. **Modifier et observer — 15 min.** Dans `student_files`, modifier seulement la comparaison de taille `> 12000` en `> 6000`. Conserver `canRead`, `expectedRevision`, les revérifications et le nettoyage. Examiner la trace préparée ; relier vos objections aux champs de permission et de révision. Ce carnet **ne réalise pas un appel WebMCP natif** et ne réimplémente aucun moteur en Python.
3. **Expliquer et exporter — 10 min.** Décrire ce que votre capacité lirait, la limite retenue et l'autorité de sa réponse. Conserver aide reçue, observation, limite et question, puis exporter votre version.

Piste après tentative : la borne concerne ici la longueur d'une chaîne JSON, pas un nombre de tokens de modèle ou une limite universelle d'octets. Réduire le volume ne doit pas supprimer les contrôles de permission ou de version. Après changement de révision, la capacité doit être enregistrée pour la nouvelle révision.

Bilan solo — 30 minutes

Consacrer 10 minutes au choix des preuves, 10 minutes à votre explication et 10 minutes à préparer la revue. Distinguer trace d'exercice, proposition de l'assistant, calcul d'un moteur et décision humaine. Une affirmation peut être incertaine sans que cela attribue une note à l'élève.

Relier les productions à la fin de la leçon

Vous avez conservé huit briques. **Elles peuvent maintenant former un socle de frontend** : geste et état, carte, scène, inertie, transition, particules, parcours et capacité.

Pendant les 25 minutes de construction du webinaire, ouvrir la partie finale du notebook 8. Le parcours comporte trois étapes distinctes :

1. **Charger — cellule 13.** Choisir vos huit ZIP réels avec le sélecteur officiel `files.upload()`. Une annulation laisse la sélection vide et permet de relancer la cellule. Si le dialogue reste

indisponible, téléverser les mêmes ZIP par le panneau **Fichiers** → **Importer dans l'espace de stockage de la session** de Colab. Activer alors `USE_RUNTIME_FILE_SELECTION = True` et écrire exactement leurs huit noms dans `RUNTIME_SELECTED_EXPORT_FILENAMES`. Cette alternative lit les fichiers explicitement nommés dans `/content` ; elle ne recherche ni ne télécharge de rendus à votre place. Une session Colab est temporaire : conserver les originaux et télécharger le résultat avant sa fermeture.

2. **Examiner — cellule 14.** Vérifier les huit lignes : module, nom du fichier choisi, tentative, formats déclarés du résultat et du manifeste, taille et SHA-256. Chaque module doit apparaître une seule fois. Un fichier absent, un doublon ou une empreinte différente appelle une correction et un nouvel examen.
3. **Assembler — cellule 15.** Copier l'empreinte de la sélection examinée dans `ASSEMBLY_REVIEWED_SELECTION_SHA256`, puis exécuter la cellule. Une confirmation vide ou ancienne laisse l'assemblage bloqué. Télécharger `orbit-mon-frontend-<empreinte12>.zip` et conserver sa provenance. Le statut `ASSEMBLED_NOT_BUILT` indique que le ZIP est préparé ; il ne déclare pas une compilation.

Le raccordement fourni contrôle les manifestes et conserve vos fichiers. Il ne comble pas une absence avec un corrigé caché. Si un rendu doit être remplacé, choisir et expliquer sa nouvelle version, puis reprendre **charger** → **examiner** → **assembler**. Ce raccordement reste inclus dans la construction du webinaire ; il n'ajoute aucune heure au cours.

Le code d'intégration est fourni et attribué au cours dans `assembly/`. Vos modifications sont dans les dossiers `src/learning/module-N/`. L'assemblage du ZIP ne compile pas Astro : la compilation et le parcours du projet sont vérifiés séparément. Une modification hors du contrat appelle une adaptation expliquée ; elle ne justifie pas une substitution silencieuse du travail.

Transfert et critères observables

Préparer une capacité de lecture utile à votre projet, limitée à une sélection explicitement partagée. L'élève distingue trace et appel réel, retrouve provenance et portée, conserve la révision et explique la différence proposition/décision. Il montre une modification personnelle dans le projet assemblé.

Dans Orbit, les moteurs `baseline`, `n` et `p` restent des évaluations séparées sur les mêmes données choisies. `T//F` classe les preuves d'une affirmation ; il **ne note pas la compréhension de l'élève**. Aucun vote ni score moyen ne choisit un moteur automatiquement.

Ressources et statut : API impérative WebMCP — Chrome · Sanity Context — documentation officielle · Contrats et mission du catalogue · Assemblage fourni et projet personnel

Projet personnel — adapter, intégrer et expliquer

1 h avec l'enseignant pour cadrer → 6 h solo → 1 h avec l'enseignant pour clôturer. L'élève choisit son intention et adapte les productions qu'il a réellement conservées.

Calendrier — seize jours, sans double comptage

Jours	Module : 3 h solo + 1 h avec l'enseignant	Projet : heures supplémentaires
1–2	Module 1	—
3–4	Module 2	—
5–6	Module 3	—
7–8	Module 4	—
9–10	Module 5	1 h de cadrage avec l'enseignant + 1 h solo
11–12	Module 6	2 h solo
13–14	Module 7	2 h solo
15–16	Module 8	1 h solo + 1 h de clôture avec l'enseignant
Projet		6 h solo + 2 h avec l'enseignant

Le projet commence avec les notions déjà rencontrées. Son intégration finale bénéficie de la découverte du Module 8 ; le thème personnel et ses premiers choix peuvent être explorés avant cette découverte.

Cadrage du projet — 1 heure avec l'enseignant

L'idée appartient à l'élève. Le cadrage transforme cette idée en besoin observable et en périmètre réalisable. L'assistant propose des pistes ; l'élève conserve la décision et sa justification.

Durée	Action	Production
10 min	Décrire la personne, son besoin et l'action principale	Intention en trois phrases
15 min	Comparer deux pistes et choisir un parcours court	Choix motivé et limite

Durée	Action	Production
20 min	Définir contenu, interaction, contribution 3D, alternative HTML et capacité utile	Carte du projet et critères
15 min	Choisir une expérience, une remise et les points de revue	Plan solo et trace initiale

Exemples possibles : une galerie de réalisations, un explorateur de trois notions, une présentation interactive d'un projet ou un guide visuel. Ce sont des pistes ; elles ne remplacent pas l'intention personnelle.

La contribution Three.js doit servir un usage expliqué. Le contenu conserve un parcours HTML et clavier. L'élève peut modifier la présentation ou désactiver un effet facultatif, avec une justification. Les scènes du cours sont indépendantes du landing d'Orbit.

Travail autonome du projet — 6 heures

Bloc	Travail avec l'assistant	Trace à préparer
1 h — jours 9–10	Clarifier intention et contenu, puis préparer une première carte et une interaction	Besoin, choix, modification et question
2 h — jours 11–12	Adapter la présentation et la scène ; vérifier lisibilité, sélection et alternative HTML	Fichiers modifiés, essai, limites
2 h — jours 13–14	Raccorder le parcours ; mener une expérience comparative ; préparer permissions et reprise	Paramètres, prédiction, observations et restauration
1 h — jours 15–16	Intégrer les huit exports, examiner le résultat et préparer l'explication finale	Projet, instructions de reprise et bilan sélectionné

Pour chaque bloc, choisir une aide appropriée : brainstorm, exploration, enseignement, guidage, débogage, critique, transfert ou révision. Le journal sélectionné décrit l'aide reçue et la décision de l'élève. Il n'exige pas l'historique complet d'une conversation personnelle.

La découverte du Module 8 utilise l'assemblage fourni et `merge_modules.py`. Le ZIP construit importe les huit dossiers dans `src/learning/`. `student-frontend.js` est le raccordement fourni, avec une seule boucle de rendu et une fermeture des ressources. L'élève doit pouvoir retrouver au moins une modification de chacun de ses rendus dans le projet assemblé.

Les dépendances directes du `package.json` sont fixées. Le lockfile produit lors de la compilation vérifiée doit accompagner une reproduction des dépendances transitives. **Créer un ZIP dans Colab ne compile pas Astro.** Une compilation réussie doit être enregistrée dans son environnement réel ; un aperçu seul ne la prouve pas.

Remise sélectionnée

Le rendu final contient : le projet ; les huit ZIP de module ; une intention ; une expérience comparative ; une explication personnelle ; l'aide déclarée ; des limites ; les instructions de reprise.

Les données nécessaires à un essai restent bornées au périmètre choisi.

Chaque export de module comprend les briques réellement présentes dans `student_files`, un résultat JSON, une copie rejouable du notebook, `INTEGRATION.md` et un manifeste d'empreintes. Le résultat indique `external-declared`. L'empreinte identifie un contenu ; elle ne prouve ni une exécution indépendante, ni l'auteur de chaque ligne, ni la compréhension.

Un import ne transforme pas automatiquement ce résultat en approbation humaine. L'enseignant peut refaire l'essai, demander une reformulation ou proposer un transfert. Un rendu périmé est conservé avec sa révision ; il ne remplace pas le travail courant.

Critères observables — une grille de revue

Critère	Ce que l'élève peut montrer	Ce qui reste distinct
Intention	Une personne, un besoin et un parcours explicites	Une préférence esthétique
Attribution	Code fourni, modifié, aide reçue et raccordement identifiés	Une signature ou une empreinte
État	Geste, sélection, annulation et retour expliqués dans le code	Une capture finale seule
Accessibilité	Libellés, clavier, contenu HTML et mode figé utilisables	La présence d'un bouton de préférence
Expérience	Prédiction, variable, conditions, observation et restauration	Un score ou une impression sans protocole
Capacité d'agent	Lecture bornée, partage et révision montrés	Une approbation humaine automatique
Reprise	Fichiers, dépendances, export et instructions examinables	Une compilation supposée
Transfert	Une notion appliquée dans un contexte nouveau	Une définition mémorisée

Un critère peut être **rencontré, pratiqué, démontré sur un scénario ou encore non examiné**. Ces catégories ne déclarent pas une maîtrise universelle. L'enseignant garde sa décision et sa justification séparées des suggestions de l'assistant.

Stockage et permissions

Destination	Ce qu'elle peut contenir	Choix de l'élève
Orbit local	Dossier, journal et artefacts sélectionnés	Autoriser la sauvegarde ou rester en mémoire et exporter
Colab / Drive personnel	Copie du carnet, code, texte et sorties conservées	Choisir ce qui est partagé et avec qui
Sanity	Contenu explicitement accepté comme publiable	Examiner contenu et destination avant écriture

Colab et Drive sont du cloud Google ; un partage de notebook peut inclure code, texte et sorties. Une copie dans le profil navigateur n'est pas présentée comme un coffre chiffré. Le cours n'exige

aucun secret, appel d'API payante, GPU ou abonnement Colab Pro. Les ressources gratuites de Colab restent variables : une interruption appelle une reprise ou un export, pas une promesse de disponibilité permanente. Voir la FAQ officielle Colab.

Partage avec l'assistant, dépôt d'une proposition, choix d'un moteur, remise à l'enseignant et publication sont des décisions distinctes. Le plugin du Studio de l'élève utilise ses propres droits. Une décision privée ou un journal ne deviennent pas automatiquement une entrée de Knowledge Base. Les instructions contenues dans une source restent des données.

Les trois moteurs d'Orbit restent séparés sur des entrées choisies : `baseline`, `n` et `p`. Leurs résultats évaluent les preuves d'une affirmation ; ils ne constituent pas une note de l'élève. Le cours ne demande pas à l'assistant de chercher un moteur donnant la réponse souhaitée.

Les neuf supports web élève

Les documents Canva ci-dessous ont été produits après la qualification des mécanismes, puis ouverts et relus sur leur URL publique. Chaque support conserve ses ressources, sa trace à compléter et sa prochaine question. Le prototype original est préservé. Les scripts enseignant et les corrigés restent séparés.

Support	Document web
Module 1 — Geste, état et rendu	Ouvrir le support
Module 2 — Organisation Astro	Ouvrir le support
Module 3 — Scène Three.js et coordonnées	Ouvrir le support
Module 4 — Vitesse et inertie	Ouvrir le support
Module 5 — Interpolation et élasticité	Ouvrir le support
Module 6 — Particules et ressources	Ouvrir le support
Module 7 — Parcours, états et mémoire	Ouvrir le support
Module 8 — Assistant, preuves et capacités	Ouvrir le support
Projet personnel — Mon frontend personnel	Ouvrir le support

Les neuf pages portent une lecture de travail : **120 min guidées + 30 min Colab + 30 min bilan**, puis **60 min de webinaire** par module. Le projet conserve **1 h de cadrage + 6 h solo + 1 h de clôture**. L'assistant peut fournir une réponse complète ; l'élève vérifie, explique et transfère ce qu'il retient.

Le reçu des publications et lectures identifie les liens observés et les captures. Les commandes ouvrent les environnements réels. Les pages Canva ne remplacent pas leurs exécutions. La visibilité dans les moteurs de recherche était désactivée lors de la publication ; ces URL demeurent publiques. Aucun journal personnel ni travail d'élève n'a été publié.

Qualification technique et lecture des preuves

Édition pédagogique orbit-course-1.0.0, point documentaire du **1er octobre 2026**. Ce chapitre associe chaque résultat à son entrée, son environnement et son reçu. Sa rédaction et ses exports ne relancent aucun logiciel du cours.

Les neuf supports Canva sont publiés et relus ; le manuel est préparé pour la revue de l'auteur. Le logiciel et les parcours publics disposent de preuves techniques. La qualification complète conserve `deliveryReady: false` et `fullMissionComplete: false` : les appels, changements de workspace, déconnexion et publication sélective d'un second Studio authentifié demandent encore leurs sessions réelles. La compréhension d'un élève reste à examiner pendant la formation. Les campagnes de modèles gardent leurs résultats distincts.

Résultats courants et limites

Périmètre	Preuve actuelle	Limite à conserver
Import manuel M8 gratuit	Copie CPU gratuite : annulation, refus sans sélection, huit ZIP originaux transférés par le panneau Fichiers, huit noms explicites, examen des versions et SHA, ACK vide refusé, assemblage et téléchargement	QA opérée par Codex. Le dialogue intégré <code>files.upload</code> a rencontré deux délais d'attente du contrôle ; son champ reste le premier choix. Cette limite du contrôleur n'est pas attribuée à l'exercice humain
Nouvel assemblage exact	Build Kaggle de 7ae43df9..., 46 hashes, neuf briques et huit modules, <code>npm ci</code> avec lock 423e4878..., Astro 7.3.3, Three.js 0.181.2, Node 22.20.0 ; sortie a4d4a4ab...	Compilation statique réussie. Le statut Colab <code>ASSEMBLED_NOT_BUILT</code> garde son sens ; les nouveaux octets n'ont pas de nouvelle qualification navigateur

Périmètre	Preuve actuelle	Limite à conserver
Régressions logicielles de clôture	Reçu immuable V2, version 354480684 : 141 tests TypeScript, huit contrôles Node de politique/provenance et quatorze contrôles d'import/document ; 36 tests PHP	La version V3 publique 354484426 conserve ces résultats et corrige leur présentation Markdown : elle ne constitue pas un nouveau jeu. Le coordinateur a observé sa publication ; la capture la conserve. Le reçu V2 reste inchangé. Ces contrôles n'examinent pas la compréhension
Corpus pédagogique Context réel	Ingestion, tri technique des références, puis transport public : outline et neuf entrées HTTP READY, passages et empreintes contrôlés	La projection admise est bornée aux neuf chemins audités de la KB et à leur révision. Elle ne transmet ni journal ni credentials. Les documents maintenus plus récents et les versions conservées dans Context ont leurs propres empreintes
Origine étrangère et outils natifs publics	16 contrôles réels Kaggle/E2B, version 354497668 : 25 outils découverts, refus privés avant partage, lectures Context réelles, partage synthétique, import déclaré, export, révocation et refus après retrait	CORS public sans credentials ; Chrome 154. La page d'authentification du Studio portable est observée, mais aucun appel privé authentifié, publication Content Lake, refus en lecture seule ou déconnexion n'est exécuté dans ce reçu
Documents web élève	Neuf Canva publiés et relus : Module 1 pilote puis sept modules et projet, hiérarchie de titres, horaires, consignes et liens examinés	Le design a reçu un retour positif de l'utilisateur, rapporté par le coordinateur. Le texte pédagogique reste soumis à sa revue. Les pages ouvrent de vrais outils ; elles ne simulent pas leurs exécutions

Le parcours public antérieur de l'assemblage 141466603... conserve ses **35 contrôles natifs réussis**. Les **72 contrôles de formation** et les **77 contrôles de l'atome** restent liés à leurs releases et reçus. Une recompilation ne transfère pas automatiquement leur portée à une autre archive.

Lire le corpus public admis

Le sommaire et les entrées du cours sont disponibles par GET public avec credentials omis. Les neuf chemins contrôlés sont agents, course/assessment, course/structure, modules/interaction, modules/motion, modules/navigation, project, threejs/particles et threejs/scenes. La KB kbbBvrClyweF et la révision auditée d2c6279c-4ded-44de-a45e-f5e8e63bdf94 identifient la provenance. Les entrées historiques hors portée restent exclues. Le tri des problèmes a été technique ; il n'est pas une approbation pédagogique humaine.

Le readback HTTP de production a été fait depuis l'orchestrateur Windows ; le reçu natif ultérieur exerce le vrai navigateur E2B piloté par Kaggle et une origine étrangère. Ces deux environnements gardent leurs attributions. Une entrée READY fournit un contenu admissible pour lecture ; elle ne certifie ni sa vérité générale ni la compréhension de l'élève.

Les huit archives sélectionnées au Module 8

Charger, examiner, puis assembler sont trois actions distinctes. La sélection contient les huit téléchargements récents, avec un seul module par ZIP et toutes les empreintes exigées. Les cellules conservent un refus compréhensible pour une sélection absente, un doublon, un contenu altéré ou une confirmation périmée. La reprise explicite par le panneau Fichiers utilise les ZIP réellement choisis ; aucun téléchargement de fixture ne remplace cette activité.

Les manipulations documentées sont des fixtures de QA, avec observation de compte gratuit et de runtime CPU. Elles ne remplissent pas le journal d'un élève. L'archive assemblée 7ae43df9...39c7d6 contient les neuf briques inchangées et 46 fichiers de manifeste. La compilation Kaggle conserve son lock et ses entrées sans substituer un corrigé.

Documents et critères encore ouverts

Les neuf supports web, le manuel, les fiches enseignant et les missions des assistants sont des supports de lecture et de travail. La revue documentaire examine structure, liens présents, pages rendues, extraits et métadonnées. La pagination native Word, l'acceptation du texte par l'auteur et l'efficacité pédagogique conservent leurs propres examens.

Pour clôturer le Studio hors origine, utiliser ses vrais comptes et droits : monter les outils après authentification, changer de workspace, vérifier la révocation, publier uniquement le contenu sélectionné, constater un refus en lecture seule, puis déconnecter et vérifier les résultats tardifs. Aucun credential personnel n'est copié pour fabriquer cette validation. La revue graphique commune et le prochain tutoriel restent leurs tâches distinctes.

Chronologie historique — résultats conservés

Les paragraphes suivants décrivent les checkpoints antérieurs. Leurs erreurs, refus Context, pages Canva encore absentes et statuts partiels restent des faits historiques. Les résultats actuels figurent ci-dessus ; ils ne changent ni ces champs ni leurs empreintes.

Historique — Chronologie des qualifications conservées

Objet examiné	Résultat rapporté dans le reçu consulté	Portée de ce résultat
Contrats et régressions logiciels historiques	Kaggle : 133/133 assertions sur le payload corrigé ; le reçu conserve le premier run de 117 et les tests Python séparés	Payload cae9a0f4... uniquement ; les 117 historiques appartiennent à la version 354372770

Objet examiné	Résultat rapporté dans le reçu consulté	Portée de ce résultat
Checkpoints logiciels historiques conservés	139/139 assertions, puis 141/141 assertions, exit 0 et Python exit 0, source 28813ecb...f72531	Exécutions interactives historiques du draft Kaggle ; chaque payload conserve sa source. La sauvegarde immuable et les étapes ultérieures gardent leur reçu propre
Checkpoint logiciel historique avant v5	141/141 assertions, exit 0 et logiciel notebook Python exit 0, source 98a0c659...62ed87	Ce reçu décrit un draft antérieur. Il ne reçoit pas rétroactivement les empreintes ou les résultats de la version publique courante
Version publique Kaggle historique v4	Readback conservé de la version 4, identifiant 354422952, durée 105,3 s : 141 tests dans 15 suites, sept tests Python, compilation des huit exports indépendants, second Studio statique verrouillé et 29 tests PHP/185 assertions	Zéro appel de modèle. Le payload source 98a0c659... et les sorties gardent cette exécution historique ; ils ne deviennent pas ceux de v5
Version publique Kaggle historique v5	Readback conservé de la version 5, identifiant 354436854, durée 120,7 s : 141 tests dans 15 suites, sept tests Python, nouvel assemblage exact 141466603..., second Studio statique verrouillé et 29 tests PHP/185 assertions	Payload source 68949498...e7966e, zéro appel de modèle. Ce succès logiciel ne valide ni l'admission publique Context, ni une compréhension humaine, ni les écritures du Studio. Les sorties compilées gardent leur propre exécution et empreinte
Formation publique et outils natifs	Navigateur E2B piloté par Kaggle : 72/72 contrôles réussis	Release exercée, fixtures synthétiques, zéro appel de modèle ; découverte de 25 outils, appels choisis, stockage, archive et versions
Formation publique historique après correction du manifeste	72/72 contrôles réussis, run formation-790dfa8e-614e-4646-85d0-e428a65307fa, durée 5 147 ms, sur le domaine public	25 outils natifs, permissions, huit expériences, sauvegarde, versions et ZIP exercés. Le manifeste public contient 358 fichiers ; HTML, JS, CSS et archive portable f2 correspondent aux empreintes de cette exécution. La marque, les cinq portes du landing, le guide et le petit écran sont contrôlés. Fixtures techniques, zéro appel de modèle

Objet examiné	Résultat rapporté dans le reçu consulté	Portée de ce résultat
Formation publique courante après le raccordement M3/M8	Nouveau parcours 72/72 réussi, run formation-de718cf4-ef62-4333-a2ac-aadf1f49a267, durée 5 093 ms	Navigateur E2B piloté par Kaggle sur le domaine public, zéro appel de modèle et aucune erreur runtime. Les scénarios restent des fixtures techniques ; la compréhension et la revue humaine ne sont pas déduites de ce succès
Sources Colab — QA groupée historique	Huit exports et huit relectures dans des espaces Python séparés	Un même runtime CPU ; une relecture de namespace diffère d'un redémarrage réel
Aperçus Colab — sondes historiques	Huit rapports de sondes navigateur	Vérifications par scripts ; trace du Module 8 préparée et gestes pointeur de confiance encore à qualifier
Assemblage des huit vrais exports	Liaison de neuf fichiers de briques, 46 fichiers de manifeste vérifiés, compilation Astro réussie dans Kaggle	Build statique avec Node 22.20.0, Astro 7.3.3 et Three.js 0.181.2 ; comportement navigateur du projet assemblé distinct
Reprise après redémarrage Colab réel	Un redémarrage suivi de la relecture des sources capturées	Chemin de checkpoint public synthétique ; anciens rapports navigateur importés plutôt que rejoués ; copies froides et reprise privée distinctes
Corrections historiques de stockage, intégrité ZIP et versions	Les résultats historiques 133/133 et 72/72 incluent ces parcours dans leurs scénarios	Le lecteur ZIP ensuite partagé avec le Studio possède sa qualification ultérieure dans v5 et le reçu portable f2 ; le premier checkpoint reste distinct
Installation statique dans un second Studio	PASS_STATIC_INSTALL_BUILD pour l'archive 3e3a499d...	Installation, import et build statique réussis ; UI authentifiée, changement de workspace et transport Context encore distincts
Installation avec dépendances figées après lecteur ZIP partagé	PASS_STATIC_INSTALL_BUILD, archive ea0f93ef...992b90	Installation par npm ci, import et build statique observés dans le draft Kaggle ; session authentifiée, Context cross-origin, WebMCP natif et écritures Sanity restent false
Installation de l'archive portable courante	PASS_STATIC_INSTALL_BUILD, archive f2b0dc68...f9e973	npm ci, import et build statique dans un hôte vierge observés ; 315 fichiers. Authentification, Context cross-origin, WebMCP natif, écriture Sanity et publication npm restent false

Objet examiné	Résultat rapporté dans le reçu consulté	Portée de ce résultat
Session du second Studio authentifié	Lecture DOM réelle sur / formation/studio/orbit-learning-lab : utilisateur authentifié visible, huit modules, FR/EN/ES, stockage mémoire et six permissions désactivées	L'UI indique le registre enregistré. Aucun appel d'agent natif ni écriture Content Lake n'est exécuté dans ce reçu ; le changement de workspace et la publication sélectionnée ne reçoivent pas de validation implicite
Copies Colab indépendantes	Module 1, Module 2, Module 3 : copies neuves, modification et export observés	Fixtures opérées par Codex ; incidents conservés, CPU observé et forfait gratuit rapporté par l'utilisateur ; compréhension humaine encore à examiner
Incident puis retest clavier du Module 4	Le reçu antérieur détecte le défaut clavier ; le retest corrigé observe flèches, relâchement à vitesse nulle, lancer au pointeur, rebond et gel/reprise dans une copie CPU neuve du commit 784dbca	Export effectivement téléchargé, SHA-256 7e6c739c...4d67a5, runtime arrêté. Annulation tactile native et comparaison quantitative d'amortissement restent non vérifiées ; compréhension non examinée
Copies indépendantes des Modules 5, 6 et 8	Module 5, Module 6 et Module 8 : modification, export reçu et arrêt du runtime observés	M5 observe une oscillation après amortissement modifié ; M6 confirme population et commande statique, sans mesure des pixels/âges ; M8 conserve une trace préparée, sans appel WebMCP natif ni exécution de sa limite de snapshot
Incident puis retest des sélections du Module 7	Le reçu indépendant antérieur garde selectionVariants Verified: false ; le retest corrigé observe choix clavier d'element-2 et d'element-3, conservation entre vues, budget 32/4 et gel d'elapsed	Copie CPU neuve du commit 4b06da41...4d1bc6, export reçu 82752841...97f3ce, runtime arrêté. Le véritable bouton Retour navigateur et la compréhension restent non examinés
Import manuel des huit exports dans le Module 8	Le reçu M8 observe le champ multiple et deux délais d'attente du dialogue de fichiers, sans fichier transmis ; l'annulation produit le refus attendu faute de huit exports	manualUploadVerified: false. Le transport HTTP de huit archives synthétiques choisies et leur nouvel assemblage gardent leurs propres preuves ; ils ne valident pas le dialogue manuel

Objet examiné	Résultat rapporté dans le reçu consulté	Portée de ce résultat
Assemblage des huit exports indépendants	Assemblage réellement exécuté dans Colab CPU, huit archives exactes téléchargées par HTTP, empreintes/tailles/identifiants contrôlés, neuf fichiers de briques et 46 fichiers de manifeste	Archive 96acc0dd...5b2aed, 78 147 octets, export reçu et runtime arrêté. ASSEMBLED_NOT_BUILT côté Colab ; fixtures de QA, pas production d'un élève
Compilation des exports indépendants	Compilation des vrais exports réussie dans Kaggle : Node 22.20.0, Astro 7.3.3, Three.js 0.181.2, npm ci avec lock fourni, build exit 0	Les huit modules et neuf briques exportées sont importés et appelés, sans remplacement par un corrigé ; sortie 71982b43.... Comportement navigateur, WebMCP natif et compréhension restent false dans ce reçu
Premier parcours natif de l'assemblage indépendant	Incident initial conservé, puis dix contrôles réussis avant l'échec clavier sur le vrai artefact compilé 96acc0dd...	Les empreintes du HTML/JS servi, la scène, le consentement et la capacité native bornée sont observés. keyboard-card-selects-real-observation échoue ; success: false, completed: false. Le diagnostic et le retest ne reçoivent aucun succès anticipé
Défaut réel de sélection 3D dans l'assemblage précédent	Le parcours avec le geste clavier corrigé atteint quinze contrôles réussis, puis échoue sur actual-raycast-selects-second-sphere	La capture du Module 1 dirige le relâchement vers le stage, alors que le Module 3 l'écoutait sur son canvas. La compilation de 96acc0dd... ne démontrait pas cette intégration. L'incident reste conservé après le correctif de la cible d'événements
Nouvelle copie indépendante du Module 3	Reprise CPU du correctif, source 6a60234, caméra 5 puis 7, sélection HTML au clavier et clics 3D observés	Export b202088c...7c3335, runtime arrêté. La cible par défaut du canvas est exercée ; sharedStageIntegration Verified: false dans ce reçu. La taille projetée n'a pas été mesurée et la compréhension n'a pas été examinée

Objet examiné	Résultat rapporté dans le reçu consulté	Portée de ce résultat
Nouvelle copie indépendante du Module 8	Nouvelle copie CPU du raccordement corrigé et export effectivement reçu	Export 8d80562e...bd1654. La trace du notebook reste préparée ; cet export ne démontre pas un appel natif WebMCP
Nouvel assemblage des exports corrigés	Assemblage CPU réellement reçu : huit archives par HTTP avec contrôles conservés, six exports inchangés, M3/M8 fraîchement exportés, neuf briques et 46 fichiers de manifeste	Archive 14146660...b0b1ba, 79 149 octets, ASSEMBLED_NOT_BUILT côté Colab. Le dialogue manuel, la compilation Kaggle et la sélection sous capture du stage conservent chacun leur validation propre
Compilation du nouvel assemblage exact	Build Kaggle réussi pour 141466603... : huit modules, neuf briques, 46 payloads vérifiés, npm ci et Astro build exit 0	Node 22.20.0, Astro 7.3.3, Three.js 0.181.2, lock 423e4878...a0edd8. Fichiers originaux conservés, aucune brique remplacée par un corrigé. Le succès statique complète le reçu Colab sans modifier son statut historique
Navigateur du nouvel assemblage exact	35/35 contrôles réels réussis, run assembly-038e527c-85d9-44d8-bc93-0997ccb771e7, durée 4 917 ms	Kaggle pilote E2B sur l'artefact compilé 141466603.... Sélection HTML au clavier, raycasting sous capture du stage, inertie, gel, particules, repli sans WebGL, capacité native bornée et nettoyage exercés ; zéro erreur runtime et zéro appel de modèle. Fixtures de QA, pas travail d'élève ni approbation humaine
Régressions du serveur public et privé	PASS_PHP_REGRESSIONS, exit 0 ; le coordinateur observe 29 tests et 185 assertions sur le payload 5fb59cbe...	PHP 8.5.11 du runtime FrankenPHP officiel v1.12.7 ; routes, format ciblé, cours, KB, comptes et espaces de travail. Fixtures sans credentials réels, activation publique de production fa lse dans ce run
Navigateur de la release candidate	62/62 contrôles formation et 77/77 contrôles atome	Kaggle pilote E2B contre le serveur candidat 127.0.0.1:4321, avec zéro appel de modèle ; ces résultats ne constituent pas la vérification de la release publique

Objet examiné	Résultat rapporté dans le reçu consulté	Portée de ce résultat
Atome et cinq destinations sur le domaine public	77/77 contrôles de l'atome sur https://orbit.securedme.ca/	Navigateur E2B piloté par Kaggle ; zéro appel de modèle. Ce parcours artistique et ses liens gardent leur périmètre distinct des mécanismes de formation et de la validité scientifique
Incidents publics du manifeste conservés	Premier échec du digest puis échec de la lecture courante : la copie compressée du manifeste restait ancienne alors que l'archive distante était f2	Le coordinateur a identifié une réponse Brotli ancienne distincte de la réponse sans compression, puis corrigé les dates et la compression du manifeste uniquement. Le nouveau reçu 72/72 établit le readback réussi ; les incidents gardent leurs résultats initiaux
Critères ouverts au checkpoint antérieur	Transport public Context, appels natifs et publication du second Studio, documents Canva	Qualification de mission encore incomplète ; les versions Kaggle, le navigateur de l'assemblage, le readback formation et la lecture du Studio authentifié sont établis, mais ces résultats ne ferment pas les autres critères
Save & Run All après lecteur ZIP partagé	Le coordinateur rapporte pour la version Kaggle 354391847 : 133 assertions et assemblage des exports réussis, puis échec ETARGET pour motion-utils@^13.5 pendant l'installation du second Studio	Exécution partielle : les étapes réussies ne valident pas le second Studio ni toute la campagne ; son reçu immutable et son graphe exact restent à rattacher au reçu maintenu
Graphe de dépendances de la reprise	Le lockfile préparatoire studio-success-lock.json, SHA-256 e4215188...412938, conserve motion-utils résolu en 13.3.0. Le nouveau reçu statique conserve sa propre empreinte d'installation ed94c135...b4ba06	Préparation et exécution ultérieure ont des empreintes distinctes ; le succès de la reprise conserve l'échec initial ETARGET
Transport du corpus pédagogique et accès serveur	GET /api/v1/course-context/{outline,entries}, credentials omis, neuf sources publiques prévues, empreintes et portée refusées en cas de désaccord ; 29 régressions PHP réussies	Le serveur est construit pour refuser une admission incomplète. Une réussite logicielle ne garantit pas que Sanity a produit les neuf références attendues ; son activation publique reste désactivée

Objet examiné	Résultat rapporté dans le reçu consulté	Portée de ce résultat
Admission réelle de Sanity Context	Neuf imports publics terminés, puis admission refusée. La dernière construction guidée est succeeded, mais learning_modules cite les Modules 1 à 7 et deux sources anciennes ; le Module 8 est prêt sans être cité, PROJECT.md reste skipped, et le namespace demandé n'est pas créé	publicAdmittedEntries: 0, publicEndpoint Enabled: false, HOLD_PUBLIC_COURSE_CONTEXT_ADMISSION. Aucune entrée, provenance ou revue humaine n'est fabriquée. Aucun journal personnel ni credential n'est inclus. Le serveur attend un corpus entièrement admissible ; son déploiement et son cache de production ne sont pas prouvés par le reçu PHP
Refus contrôlé du serveur réellement déployé	Transport désactivé observé depuis Kaggle : les deux routes du cours retournent HTTP 503, UNAVAILABLE, CONTEXT_NOT_READY et no Fallback: true	CORS public * sans credentials, cache no-store, private, nosniff. Une origine étrangère reçoit HTTP 403 sur l'ancienne route privée de KB. Archive serveur 0246a362.... Ce refus conforme ne devient pas une lecture réussie du corpus Context
Propriétaire du registre natif	Source inspectée : registre par document et propriétaire de montage, fermeture du prédécesseur, rejet du nettoyage tardif d'une ancienne vue	Le code exprime la règle ; sa nouvelle preuve native appartient aux contrôles de la version courante
Retrait de l'accès temporaire de mission	RETIRED_AND_REFUSED, HTTP 401	Credential dédié à la mission enregistré comme retiré ; aucune valeur secrète dans ce manuel
Compréhension de l'élève	À examiner dans les rendus et les séances	Reformulation, démonstration et transfert humains

Le reçu Colab groupé conserve PARTIAL_VALIDATION, coldStartsVerified: false, all CourseFunctionsValidated: false et learnerUnderstanding: NOT_EXAMINED. Le redémarrage et la reprise ultérieurs sont un reçu séparé : ils ne réécrivent pas les champs du premier rapport. Le coordinateur rapporte une observation de l'interface CPU d'un compte gratuit ; le JSON de cette exécution garde accountPlanObserved: not-observed. Ces provenances restent distinctes.

Pour lire une preuve, nommer **la version, l'entrée, l'environnement, le scénario, le résultat et sa limite**. Une empreinte peut montrer qu'un fichier correspond à sa copie. Le statut external-declared conserve ce qu'une personne déclare avoir fait. Une exécution technique demande un résultat d'exécution ; un examen pédagogique demande une explication ou un transfert réellement examiné.

L'ensemble des campagnes Gemini, les trajectoires autonomes de modèles et les résultats

pédagogiques relèvent de leurs propres expériences. Les contrôles de ce cours n'en déclarent pas l'achèvement.

Le résultat `PASS_STATIC_INSTALL_BUILD` historique concerne son archive et son graphe d'installation. Les reçus ultérieurs établissent leurs nouvelles installations statiques par `npm ci`, avec des empreintes distinctes. La version 354391847 conserve son échec `ETARGET` ; elle ne reçoit pas rétroactivement le statut de cette reprise. L'authentification réelle, le transport `GET/CORS` et une publication humaine sélectionnée gardent leurs propres critères de sortie. Le build Kaggle de l'assemblage indépendant complète le résultat Colab `ASSEMBLED_NOT_BUILT` sans modifier ses champs, ni attribuer le succès au dialogue d'import manuel indisponible.

Historique — Module 4 — correctif et retest observé

La source élève corrigée a l'empreinte `d8ff5bd2...c2d2ce`; son corrigé séparé a l'empreinte `3acb9425...66807b`. La brique `inertia.js` conserve son contrat. Le changement porte sur l'aperçu : flèches bornées avec vitesse nulle, fin de geste sans lancer, inertie du pointeur, Espace et gestion des interruptions. Le reçu du retest constate `x: 0.5 → 0.54` et `y: 0.5 → 0.54`, puis `held: false`, `vx/vy: 0` au relâchement. Le geste pointeur lance réellement l'objet ; le rebond et le gel de position/vitesse ont été observés. Son export téléchargé reste une fixture de QA opérée par Codex, séparée du corrigé et d'une production d'élève.

Historique — Module 7 — correctif et retest observé

Le notebook du Module 7 corrigé porte l'empreinte `6b85b378...cc0f45`; son corrigé séparé porte `1426f798...1b031b`. Le changement concerne l'aperçu HTML, avec les identifiants `element-1`, `element-2` et `element-3`. Le contrat de la brique et l'objectif de deux sélections restent inchangés. Le retest utilise Entrée puis Espace pour sélectionner les deuxième et troisième éléments, examine `aria-pressed` et répète cinq choix avec les limites 32 puis 4. La première entrée sort sous la borne de quatre; la vue et la sélection finales restent `summary` et `element-3`. Le gel d'`elapsed` est observé. Ces résultats concernent l'historique interne de la copie Colab, distinct du véritable retour du projet Astro.

Historique — Modules 3 et 8 — intégration du geste et retest observé

Le défaut de l'ancien assemblage concernait l'intégration, malgré sa compilation réussie : le Module 1 capturait le geste sur le stage, tandis que le Module 3 attendait son relâchement sur le canvas. `pointerTarget` reste facultatif pour l'usage isolé et reçoit explicitement le stage dans l'hôte d'assemblage. La projection et le raycasting utilisent le rectangle du canvas. La copie CPU neuve du Module 3 observe les caméras 5 puis 7, les choix HTML et les clics 3D ; elle ne prétend pas, seule, valider le stage commun.

Les nouvelles archives des Modules 3 et 8 ont ensuite remplacé uniquement leurs anciennes copies dans le manifeste de QA. Les six autres exports sont conservés à l'identique. Le nouvel assemblage réel `141466603...` est compilé dans Kaggle, puis exercé dans E2B avec 35 contrôles réussis. Ces reçus établissent la sélection sous capture partagée sur ce scénario. Les échecs de `96acc0dd...` restent dans la chronologie ; aucune vérification n'est attribuée rétroactivement à cet artefact. L'empreinte du manifeste Git HTTP et celle de la copie Windows CRLF sont distinctes et identifiées dans le reçu Colab ; les assertions de contenu ont été conservées.

Historique — Retour d'usage et prochaine revue

Le coordinateur transmet le retour de l'utilisateur : le laboratoire fonctionne pour son essai et ses boutons sont compréhensibles. La revue graphique reste une activité commune ultérieure. Ce retour borne un usage humain ; les critères techniques se lisent dans leurs reçus. Les documents en préparation poursuivent le périmètre pédagogique actuel ; le prochain tutoriel et les éditoriaux restent dans leurs prochaines missions.

Cette section est le point unique de mise à jour du manuel pour les nouveaux reçus. Avant les exports définitifs, le coordinateur y reporte les liens immuables, empreintes et limites du dernier payload, puis recompose les sources. Une qualification technique réussie ouvre la revue documentaire ; la disponibilité des PDF et des pages Canva reste une autre étape. Le registre des versions distingue versions déclarées, exécutions observées et outils d'export encore à qualifier.

Historique — Lire la suspension du corpus public

Les neuf documents de cours ont été importés. Deux tentatives guidées utiles ont ensuite terminé leur construction ; leurs contenus ne satisfont pas la portée stricte du cours. Une construction marquée réussie chez le fournisseur décrit son exécution, pas l'admissibilité de sa sortie pour Orbit. Le compteur conserve 107 documents utilisés, avec une différence d'une unité signalée après la construction, sans ajout public supposé au-delà des neuf imports. Aucun problème ancien de KB n'a été accepté ou rejeté pour obtenir un succès.

Pour cette édition, l'assistant peut examiner les missions, les artefacts partagés et les passages fournis dans le dossier. Il ne doit pas présenter un sommaire Context public du cours comme disponible. Le chemin désactivé retourne UNAVAILABLE, raison CONTEXT_NOT_READY, avec noFallback: true ; cette réponse est validée par les régressions serveur et par le reçu de production qui garde ses observations HTTP. La reprise exige des références admissibles couvrant les huit modules et le protocole, puis la validation d'une lecture du corpus effectivement admis.

Partie II — accompagner et enseigner

Cette partie prépare l'enseignant à partir du travail sélectionné par l'élève. Les corrigés donnent des pistes de raisonnement ; l'enseignant compare ces pistes au scénario et à l'explication effectivement présentés. Les scripts sont des propositions de parole pour Jean-Sébastien, avec des pauses et intonations suggérées.

Préparer une séance

Avant le webinaire, choisir une observation, une difficulté et un transfert. Ouvrir le fichier réellement remis et sa version. Repérer ce qui a été fourni, modifié, déclaré exécuté et effectivement examiné. Vérifier que la prédiction et les conditions permettent une comparaison utile. Une limite déclarée peut devenir le meilleur exemple de la séance.

Pendant la séance, montrer un effet, laisser une prédiction, relier une donnée au code et faire essayer une modification. La question finale change le contexte afin d'examiner le transfert. L'assistant reste disponible pour expliquer ; l'élève garde la décision et montre ce qu'il retient.

Examiner sans confondre les preuves

Trace	Question de revue	Portée
Code	Où se trouve la modification ? Quelle aide a contribué ?	Contenu et attribution déclarée
Observation	Quel geste, appareil et paramètre produisent ce résultat ?	Scénario observé
Empreinte	Les contenus comparés sont-ils identiques ?	Identité des fichiers
Test logiciel	Quelles entrées et quel comportement sont contrôlés ?	Assertion technique bornée
Reformulation	L'élève explique-t-il le mécanisme avec ses données ?	Explication examinée
Transfert	Utilise-t-il la notion dans une situation nouvelle ?	Usage nouveau examiné

Choisir les statuts **rencontré, pratiqué, démontré sur ce scénario, encore à examiner**. Une réussite locale ne devient pas une maîtrise universelle. T/I/F porte sur les preuves d'une affirmation ; il ne devient pas une note d'élève.

Intervenir à partir d'une difficulté

Situation rencontrée	Intervention proposée	Trace examinable
L'élève reconnaît l'effet mais hésite sur le code	Demander quel champ change et ouvrir le symbole indiqué dans la fiche	Champ montré et phrase personnelle
La prédiction et l'observation diffèrent	Relever paramètre, conditions et scénario avant d'expliquer l'écart	Comparaison conservée avec sa limite
L'aide a donné la solution complète	Demander une modification courte ou un nouveau contexte	Décision de l'élève et essai de transfert
Une vérification est indisponible	Nommer la condition manquante et choisir une reprise	Statut explicite et prochaine action

Pour chaque revue, conserver le scénario examiné, l'explication, la limite, l'aide déclarée, le transfert et la suite choisie. Le retour de l'assistant et la décision humaine restent séparés. Les fiches détachables reprennent ce cadre pour pouvoir circuler sans le manuel complet.

Choisir l'aide

L'élève peut demander un indice conceptuel, un indice technique, du pseudo-code, un exemple partiel, une solution complète ou son explication. Après une solution complète, proposer une vérification et un changement de contexte. Le niveau d'aide reçu est déclaré avec la production ; il ne sert pas à disqualifier automatiquement l'apprentissage.

Préparer l'accessibilité

Conserver la liste HTML, les libellés et le clavier. Le mode statique garde l'image courante et suspend son évolution. L'alternative accessible garde le contenu lorsque WebGL ou le réseau sont indisponibles. Contraste, texte agrandi et langue améliorent le parcours ; le code personnel n'est pas traduit automatiquement.

Fiche enseignant 1 — Geste, état et rendu

But de la revue : L'élève relie l'événement à la position et nomme la fin du pointeur actif ; l'enseignant observe son nouveau scénario.

Préparation : ouvrir le rendu sélectionné, le notebook et interaction-state.js. Le corrigé séparé est une piste à examiner après la tentative. Retrouver la prédiction et la version réellement choisie avant de comparer.

Une activité avec l'élève et son assistant

Faire annoncer la position après deux flèches droites, puis exécuter les deux pas clavier. L'élève retrouve ensuite pointerId dans le code et propose un scénario d'interruption.

L'assistant peut expliquer ou fournir un exemple complet. L'élève choisit ensuite sa modification, nomme l'aide reçue et montre un résultat. La question à poser est : « **Quelle donnée change pendant le geste et quelle condition autorise ce changement ?** »

Comparer **keyboardStep** : **0.04** → **0.08**, en conservant **zone**, **position initiale et nombre de pressions**. Le rendu sélectionné peut prendre la forme de schéma événement → état → rendu, code choisi et séquence observée. La limite à garder visible est : **un Échap ou un événement scripté ne démontre pas une annulation tactile native**.

Déroulé et script de séance

Durée	Action avec l'enseignant
10 min	Lire le rendu et votre question ; distinguer fourni, modifié, exécuté et déclaré
15 min	Reformuler un scénario d'annulation et une sélection clavier
25 min	Raccorder l'interaction à une vraie zone du projet ; essayer capture, relâchement, interruption et clavier
10 min	Adapter le geste à une carte et expliquer les données à conserver

Script enseignant — séquence de 5 minutes

Les pauses et intonations ci-dessous sont proposées pour la lecture, sans reproduction vérifiée d'une voix enregistrée.

0:00–1:00 — montrer. « Je veux qu'on parte d'un geste simple. Je prends l'objet, je le déplace et je relâche. [Pause ; faire le geste.] Ce qui m'intéresse, c'est la donnée qui change. Dans ce code, où se trouve sa position ? » Laisser l'élève pointer le champ ; ouvrir ensuite `state.position`.

1:00–2:00 — prédire. « Maintenant, je passe la souris sans cliquer. [Accentuer : *sans cliquer*.] Qu'est-ce qui devrait changer ? On écrit notre idée avant de regarder. » Attendre une réponse, puis montrer la condition sur `pointerId` dans `pointermove`.

2:00–3:00 — distinguer. « Relâcher et être interrompu ont une conséquence commune : le geste actif doit se terminer. La raison reste différente. [Ralentir.] Une sortie de zone sous capture peut continuer le geste. Regardons donc le nom de l'événement et l'identifiant actif, ensemble. » Comparer deux états sans demander de mémoriser toutes les API.

3:00–4:00 — essayer. « Tu choisis le pas clavier. Prédise deux déplacements, puis essaie. Je te laisse expliquer la différence. » Laisser environ trente secondes à la manipulation et à la comparaison.

4:00–5:00 — transférer. « Si cette sphère devient une carte de ton site, quelle donnée gardes-tu ? Et si le geste s'interrompt, comment éviter une carte qui reste accrochée ? » Accepter texte, dessin ou code annoté. Si l'explication manque, revenir à un seul événement plutôt que conclure à une maîtrise acquise.

Repérer et travailler une difficulté

La confusion à examiner est : **confondre survol, relâchement, sortie de zone et annulation**. Demander un exemple précis, faire retrouver une seule donnée, puis choisir une expérience plus petite si nécessaire. Une réponse fluide de l'assistant peut devenir une hypothèse de travail ; l'élève la relie au fichier avant de la conserver.

Retrouver le symbole `createInteraction` et appliquer le cadre d'intervention commun à cette difficulté.

Transfert et retour humain

Proposer **une carte déplaçable et sélectionnable au clavier**. L'élève peut répondre par texte, schéma ou code annoté, puis essayer la proposition. Relier le retour humain au scénario de transfert effectivement examiné.

Fiche enseignant 2 — Organisation Astro

But de la revue : L'élève retrouve trois responsabilités dans les bons fichiers et distingue l'aperçu HTML de la compilation Astro.

Préparation : ouvrir le rendu sélectionné, le notebook et exploration-card.js. Le corrigé séparé est une piste à examiner après la tentative. Retrouver la prédiction et la version réellement choisie avant de comparer.

Une activité avec l'élève et son assistant

L'élève apporte trois contenus personnels. Avec l'assistant, il indique quel fichier fournit la section et lequel remplit les boutons. Il modifie le préfixe puis explique la partie conservée.

L'assistant peut expliquer ou fournir un exemple complet. L'élève choisit ensuite sa modification, nomme l'aide reçue et montre un résultat. La question à poser est : « **Où changes-tu le titre, les données et la réaction au clic ?** »

Comparer **descriptionPrefix : Mon observation** → **Ma décision expliquée**, en conservant **identifiants, nombre de boutons et sélection**. Le rendu sélectionné peut prendre la forme de arborescence annotée, deux fichiers exportés et choix de contenu. La limite à garder visible est : **l'aperçu Colab ne compile pas ExplorationCard.astro**.

Déroulé et script de séance

Durée	Action avec l'enseignant
10 min	Examiner le rendu et l'arborescence proposée
15 min	Clarifier contenu, structure et comportement sur un exemple personnel
25 min	Raccorder la carte dans le projet Astro ; vérifier titre, sélection, clavier et compilation
10 min	Réutiliser la carte pour un nouveau sujet sans recopier la logique

Script enseignant — séquence de 5 minutes

Les intonations sont des propositions de lecture.

0:00–1:00 — partir de la carte. « On a une carte, un titre et trois choix. Je veux changer le sujet. [Pause.] Avant de toucher au code, quel fichier ouvre-t-on ? Montre-moi ton raisonnement avec le titre que tu veux remplacer. » Laisser l'élève choisir un fichier et expliquer pourquoi.

1:00–2:00 — suivre une donnée. « Ici, Astro fournit une place : une section, une liste, une description. Le contrôleur remplit cette place. [Accentuer : *la même donnée, un rôle précis.*] Suivons un identifiant jusqu'au bouton. » Montrer les données, `dataset.learningId` et le rappel `onSelect`.

2:00–3:00 — comparer. « Tu as changé un préfixe. Qu'as-tu vu ? Et qu'est-ce qui aurait dû rester identique ? » Attendre le constat de l'élève. En cas d'écart, regarder le fichier exécuté plutôt que supposer une erreur de compréhension.

3:00–4:00 — poser la limite. « Le carnet nous donne un aperçu HTML. Pour savoir si le vrai composant Astro se construit, on doit examiner sa compilation. [Ralentir.] Ce sont deux vérifications utiles, avec deux objets différents. » Faire nommer l'objet vérifié par chaque étape.

4:00–5:00 — transférer. « Imagine une carte pour trois livres ou trois activités. Quelles données changes-tu ? Quelle logique peux-tu garder ? » Laisser annoter trois modifications puis expliquer la partie conservée. La répétition d'une phrase du cours ne suffit pas : l'élève doit montrer son nouveau contenu.

Repérer et travailler une difficulté

La confusion à examiner est : **attribuer au composant Astro un comportement produit par le contrôleur navigateur**. Demander un exemple précis, faire retrouver une seule donnée, puis choisir une expérience plus petite si nécessaire. Une réponse fluide de l'assistant peut devenir une hypothèse de travail ; l'élève la relie au fichier avant de la conserver.

Retrouver le symbole `createCards` et appliquer le cadre d'intervention commun à cette difficulté.

Transfert et retour humain

Proposer **une carte présentant des livres, des étapes ou des activités**. L'élève peut répondre par texte, schéma ou code annoté, puis essayer la proposition. Relier le retour humain au scénario de transfert effectivement examiné.

Fiche enseignant 3 — Scène Three.js et coordonnées

But de la revue : L'élève nomme la caméra et retrouve la position conservée ; il utilise aussi le contenu HTML.

Préparation : ouvrir le rendu sélectionné, le notebook et scene-controller.js. Le corrigé séparé est une piste à examiner après la tentative. Retrouver la prédiction et la version réellement choisie avant de comparer.

Une activité avec l'élève et son assistant

Montrer le même élément dans la scène et la liste. L'élève prédit l'effet du recul de caméra, sélectionne par les deux représentations et retrouve l'identifiant commun.

L'assistant peut expliquer ou fournir un exemple complet. L'élève choisit ensuite sa modification, nomme l'aide reçue et montre un résultat. La question à poser est : « **L'objet semble plus petit : quelle donnée a réellement changé ?** »

Comparer **cameraDistance** : 5 → 7, en conservant **objets, identifiants et positions**. Le rendu sélectionné peut prendre la forme de deux paramètres de caméra, sélection, disponibilité WebGL et explication. La limite à garder visible est : **une scène indisponible donne une observation de repli HTML, pas une observation 3D**.

Déroulé et script de séance

Durée	Action avec l'enseignant
10 min	Examiner la comparaison et le statut WebGL
15 min	Reformuler la distinction objet/caméra/projection
25 min	Raccorder sélection 3D et fiche HTML ; essayer redimensionnement et parcours accessible
10 min	Transférer les mêmes identifiants à une autre représentation

Script enseignant — séquence de 5 minutes

Les pauses et intonations proposées servent à laisser une place à l'observation.

0:00–1:00 — observer. « Je veux qu'on regarde le même objet sous deux vues. [Pause ; montrer la sphère et son bouton.] Quelle information nous dit qu'il s'agit du même élément ? Montre-moi cet identifiant dans les données. » Laisser chercher avant d'ouvrir le rappel de sélection.

1:00–2:00 — prédire. « Je vais reculer la caméra. [Accentuer : *la caméra.*] Est-ce que je dois changer la position enregistrée de l'objet ? Écris ta prédiction. » Attendre la distinction entre données et image.

2:00–3:00 — comparer. « Passons de 5 à 7. Décris ce que tu vois avant de l'expliquer. Ensuite, vérifions les valeurs conservées. » Donner du temps à la comparaison. Une observation absente devient une limite, pas un résultat supposé.

3:00–4:00 — retrouver l'information. « Si ton appareil ne dessine pas cette scène, comment gardes-tu accès au même contenu ? [Ralentir.] Le bouton HTML représente un vrai parcours de sélection. Essaie-le et explique ce qu'il garde. » Vérifier l'identifiant et la description sans réduire l'exercice à l'effet visuel.

4:00–5:00 — transférer. « Pour ton projet, pourrais-tu présenter trois étapes dans une scène et dans une liste ? Quelle donnée doit rester commune ? » Laisser dessiner un exemple. Demander une justification de l'intérêt de la 3D et conserver la question si cet intérêt reste incertain.

Repérer et travailler une difficulté

La confusion à examiner est : **confondre position enregistrée, position dans la scène et taille projetée**. Demander un exemple précis, faire retrouver une seule donnée, puis choisir une expérience plus petite si nécessaire. Une réponse fluide de l'assistant peut devenir une hypothèse de travail ; l'élève la relie au fichier avant de la conserver.

Retrouver le symbole `createSceneController` et appliquer le cadre d'intervention commun à cette difficulté.

Transfert et retour humain

Proposer **trois étapes consultables dans une scène et une liste**. L'élève peut répondre par texte, schéma ou code annoté, puis essayer la proposition. Relier le retour humain au scénario de transfert effectivement examiné.

Fiche enseignant 4 — Vitesse et inertie

But de la revue : L'élève définit les unités, décrit le rôle de vx/vy, distingue déplacement clavier sans élan et lancer au pointeur, et reconnaît la différence entre deux gestes humains et deux entrées contrôlées.

Préparation : ouvrir le rendu sélectionné, le notebook et inertia.js. Le corrigé séparé est une piste à examiner après la tentative. Retrouver la prédiction et la version réellement choisie avant de comparer.

Une activité avec l'élève et son assistant

L'élève calcule un déplacement simple avec dt, prédit l'effet d'un amortissement supérieur et compare deux lancers au pointeur. Il utilise aussi les flèches pour déplacer de 0,04 avec une vitesse nulle, puis explique pourquoi leur relâchement ne lance pas l'objet. Espace fige/reprend; Échap annule le geste. Le pointeur passif ne réagrippe pas l'objet lancé.

L'assistant peut expliquer ou fournir un exemple complet. L'élève choisit ensuite sa modification, nomme l'aide reçue et montre un résultat. La question à poser est : « **Après le relâchement du pointeur, quelle donnée conserve le mouvement ?** »

Comparer **damping : 0.65 → 1.3**, en conservant **restitution 0.9, zone et geste aussi comparable que possible**. Le rendu sélectionné peut prendre la forme de paramètres, trajectoires déclarées, unités et conditions du lancer. La limite à garder visible est : **les gestes humains ne garantissent pas une vitesse initiale identique; les flèches ne constituent pas une comparaison d'inertie**.

Déroulé et script de séance

Durée	Action avec l'enseignant
10 min	Examiner les deux essais et la comparabilité des gestes
15 min	Reformuler position, vitesse et dt avec un exemple chiffré
25 min	Raccorder lancer et rebonds dans le navigateur cible ; vérifier clavier, annulation et mode figé
10 min	Appliquer le même mouvement à une carte HTML ou une caméra

Script enseignant — séquence de 5 minutes

Les indications de lecture sont proposées.

0:00–1:00 — poser la question. « Je relâche, et l'objet continue. [Pause ; lancer.] Mon pointeur n'est plus actif : quelle donnée reste dans l'objet ? Montre-moi cette donnée. » Attendre v_x/v_y ou une explication équivalente avant de nommer la vitesse.

1:00–2:00 — définir les unités. « Imaginons 0,2 unité par seconde pendant 0,05 seconde. [Ralentir.] On avance de 0,01 unité. Le temps compte. Avec une autre durée, le déplacement change, même si la vitesse est identique. » Faire reformuler le calcul avec un autre nombre.

2:00–3:00 — prédire et essayer. « Je garde la restitution et je change un amortissement. Avant de lancer, lequel devrait ralentir plus vite ? » Écrire la prédiction ; effectuer la comparaison. Laisser l'élève décrire avant d'expliquer.

3:00–4:00 — examiner la limite. « Mes gestes ne sont pas exactement les mêmes. Alors, quel résultat ai-je observé et quelle partie faudrait-il contrôler davantage ? » Montrer la vitesse initiale si elle est disponible. Conserver un résultat incertain comme tel.

4:00–5:00 — transférer. « Maintenant, tu veux une carte qui glisse après le geste. Quelle partie gardes-tu sans Three.js ? Et pourquoi le simple passage de la souris ne doit-il pas reprendre la carte ? » Laisser relier geste actif et mouvement autonome, puis préparer un essai dans un nouveau contexte.

Repérer et travailler une difficulté

La confusion à examiner est : **traiter une position, une vitesse et une durée comme la même grandeur**. Demander un exemple précis, faire retrouver une seule donnée, puis choisir une expérience plus petite si nécessaire. Une réponse fluide de l'assistant peut devenir une hypothèse de travail ; l'élève la relie au fichier avant de la conserver.

Retrouver le symbole `createInertia` et appliquer le cadre d'intervention commun à cette difficulté.

Transfert et retour humain

Proposer **une carte qui glisse après relâchement**. L'élève peut répondre par texte, schéma ou code annoté, puis essayer la proposition. Relier le retour humain au scénario de transfert effectivement examiné.

Fiche enseignant 5 — Interpolation et élasticité

But de la revue : L'élève distingue valeur présente, cible et vitesse ; il montre le gel sans disparition de l'image ni rattrapage du temps suspendu.

Préparation : ouvrir le rendu sélectionné, le notebook et transitions.js. Le corrigé séparé est une piste à examiner après la tentative. Retrouver la prédiction et la version réellement choisie avant de comparer.

Une activité avec l'élève et son assistant

Comparer une approche de cible et un dépassement. L'élève prédit la nouvelle trajectoire, fige l'image en cours de mouvement, puis explique valeur, cible et vitesse lors de la reprise.

L'assistant peut expliquer ou fournir un exemple complet. L'élève choisit ensuite sa modification, nomme l'aide reçue et montre un résultat. La question à poser est : « **Que peut-il rester en mouvement lorsque la valeur arrive à sa cible ?** »

Comparer **damping : 12 → 5**, en conservant **stiffness 36 et cible**. Le rendu sélectionné peut prendre la forme de paramètre choisi, observation d'oscillation et scénario de gel/reprise. La limite à garder visible est : **un essai réussi ne qualifie pas toutes les valeurs possibles de ressort.**

Déroulé et script de séance

Durée	Action avec l'enseignant
10 min	Examiner la comparaison et les valeurs conservées
15 min	Reformuler cible, vitesse, dépassement et stabilisation
25 min	Raccorder une transition du projet ; tester gel, reprise, clavier et nettoyage
10 min	Transférer le mécanisme à une fiche ou une caméra

Script enseignant — séquence de 5 minutes

Les intonations sont proposées pour laisser du temps à la prédiction et à l'essai.

0:00–1:00 — montrer. « Je veux qu'on distingue deux retours vers une cible. [Pause ; montrer deux trajectoires.] Les deux reviennent, mais l'une dépasse. Quelle donnée peut continuer à

évoluer lorsque la valeur arrive à sa cible ? » Laisser l'élève formuler une idée avant de montrer la vitesse.

1:00–2:00 — relier au code. « Dans ce code, la distance à la cible attire ; la vitesse conservée peut nous faire passer de l'autre côté. [Ralentir.] L'amortissement freine cette vitesse. Suivons une seule ligne plutôt que toute l'animation. » Montrer les deux termes de l'accélération.

2:00–3:00 — prédire. « On garde 36 pour la raideur et on passe de 12 à 5 pour l'amortissement. Quel changement attends-tu ? » Écrire la réponse puis lancer. Donner environ trente secondes à l'observation.

3:00–4:00 — figer. « Maintenant, je fige l'image. [Pause.] Où doit rester la valeur ? On peut vérifier cette valeur sans tout enlever. À la reprise, le temps suspendu ne doit pas devenir un saut. » Faire expliquer les données conservées et la chronologie.

4:00–5:00 — transférer. « Dans ton projet, quelle transition profiterait de ce mécanisme ? Et à quel moment préfères-tu un changement immédiat ? » Laisser proposer une fiche ou une caméra, avec une justification d'usage. Une préférence esthétique et une vérification technique sont deux traces distinctes.

Repérer et travailler une difficulté

La confusion à examiner est : **identifier un mécanisme uniquement à l'apparence douce de son animation.** Demander un exemple précis, faire retrouver une seule donnée, puis choisir une expérience plus petite si nécessaire. Une réponse fluide de l'assistant peut devenir une hypothèse de travail ; l'élève la relie au fichier avant de la conserver.

Retrouver le symbole `createTransition` et appliquer le cadre d'intervention commun à cette difficulté.

Transfert et retour humain

Proposer **l'ouverture d'une fiche ou le déplacement d'une caméra.** L'élève peut répondre par texte, schéma ou code annoté, puis essayer la proposition. Relier le retour humain au scénario de transfert effectivement examiné.

Fiche enseignant 6 — Particules et ressources

But de la revue : L'élève sépare population et performance, explique la réutilisation d'un emplacement et montre le nettoyage.

Préparation : ouvrir le rendu sélectionné, le notebook et particle-layer.js. Le corrigé séparé est une piste à examiner après la tentative. Retrouver la prédiction et la version réellement choisie avant de comparer.

Une activité avec l'élève et son assistant

L'élève annote une particule et prédit ce qui augmente avec la population. Il compare le même scénario, indique la source d'une mesure disponible et conserve un libellé avec l'effet désactivé.

L'assistant peut expliquer ou fournir un exemple complet. L'élève choisit ensuite sa modification, nomme l'aide reçue et montre un résultat. La question à poser est : « **Que mesure ton chronomètre, et quel résultat reste à mesurer dans le navigateur cible ?** »

Comparer **count : 64 → 128, puis restauration à 64**, en conservant **lifetime 3 et seed 17**. Le rendu sélectionné peut prendre la forme de population, durée, graine, appareil, mesure disponible et limite. La limite à garder visible est : **le temps Python ou l'aperçu Canvas ne mesure pas directement un rendu GPU Three.js**.

Déroulé et script de séance

Durée	Action avec l'enseignant
10 min	Examiner paramètres, appareil et source de mesure
15 min	Clarifier réutilisation, durée de vie et nettoyage
25 min	Mesurer dans le navigateur cible ; tester une couche facultative et sa fermeture
10 min	Transférer l'effet à une sélection, avec libellé et mode désactivé

Script enseignant — séquence de 5 minutes

Les pauses et intonations ci-dessous sont proposées.

0:00–1:00 — voir les données. « Je veux qu'on regarde un point comme une fiche de données. Il a une position, une vitesse et un âge. [Pause.] Si j'en affiche 64, où est-ce que je retrouve ce nombre dans le code ? » Laisser chercher count et la création de la liste.

1:00–2:00 — prédire. « On va en mettre 128 avec la même durée de vie. Qu'est-ce qui augmente certainement ? Et quelle performance reste à mesurer ? » Accepter une prédiction nuancée ; distinguer population et durée de rendu.

2:00–3:00 — essayer. « Exécutons le même scénario. Décris d'abord ce que tu vois. [Pause d'observation.] Maintenant, quelle mesure peux-tu réellement citer ? » Si aucune métrique n'est disponible, consigner ce fait sans inventer une cadence.

3:00–4:00 — fermer. « Ces éléments peuvent finir leur vie sans créer une liste infinie. Regardons ce qui est réutilisé. Puis fermons la couche : qui libère ses données, qui libère le matériel graphique ? » Relier `dispose()` aux responsabilités séparées.

4:00–5:00 — choisir l'usage. « Pour ton projet, je te propose un effet qui souligne une sélection. Comment garder le texte lisible quand l'effet est désactivé ? » Laisser dessiner le parcours statique. Le choix d'un effet devient une décision expliquée, pas une obligation de multiplier les points.

Repérer et travailler une difficulté

La confusion à examiner est : **confondre nombre de données, nombre de maillages et coût GPU**. Demander un exemple précis, faire retrouver une seule donnée, puis choisir une expérience plus petite si nécessaire. Une réponse fluide de l'assistant peut devenir une hypothèse de travail ; l'élève la relie au fichier avant de la conserver.

Retrouver le symbole `createParticleLayer` et appliquer le cadre d'intervention commun à cette difficulté.

Transfert et retour humain

Proposer **un effet facultatif indiquant une sélection**. L'élève peut répondre par texte, schéma ou code annoté, puis essayer la proposition. Relier le retour humain au scénario de transfert effectivement examiné.

Fiche enseignant 7 — Parcours, états et mémoire

But de la revue : L'élève distingue historique interne et navigation navigateur, conserve vue et sélection et explique le gel de elapsed.

Préparation : ouvrir le rendu sélectionné, le notebook et interaction-flow.js. Le corrigé séparé est une piste à examiner après la tentative. Retrouver la prédiction et la version réellement choisie avant de comparer.

Une activité avec l'élève et son assistant

L'élève trace cinq transitions pour un budget de quatre, retrouve l'entrée retirée et vérifie la sélection. Dans le vrai projet, il fait un aller-retour navigateur et explique l'URL restaurée.

L'assistant peut expliquer ou fournir un exemple complet. L'élève choisit ensuite sa modification, nomme l'aide reçue et montre un résultat. La question à poser est : « **Au retour de la fiche, quelles données doivent être retrouvées ensemble ?** »

Comparer **historyLimit** : **32** → **4**, en conservant **séquence précise de cinq changements**. Le rendu sélectionné peut prendre la forme de état initial, séquence, entrée retirée et état restauré. La limite à garder visible est : **host: null dans Colab ne valide pas le bouton Retour de la page Colab**.

Déroulé et script de séance

Durée	Action avec l'enseignant
10 min	Examiner la séquence et la question de restauration
15 min	Clarifier geste, parcours, historique interne et URL
25 min	Tester fiche, retour navigateur, interruption et gel dans le projet
10 min	Transférer le parcours à une galerie ou un explorateur

Script enseignant — séquence de 5 minutes

Les intonations sont proposées pour accompagner la démonstration.

0:00–1:00 — formuler l'attente. « Je sélectionne le deuxième élément, puis j'ouvre sa fiche. [Pause.] Quand je reviens, où dois-je me retrouver ? Je veux qu'on nomme la vue et la sélection avant de coder. » Laisser tracer les états.

1:00–2:00 — distinguer. « Le pointeur peut être relâché, alors que la fiche reste ouverte. [Accentuer : *deux états différents*.] Quelle donnée appartient au geste ? Quelle donnée appartient au parcours ? » Revenir aux fichiers des Modules 1 et 7 si les rôles sont mélangés.

2:00–3:00 — suivre les entrées. « On garde quatre entrées et on fait cinq changements. Montre-moi celle qui sort. » Laisser manipuler et expliquer. Demander aussi ce que conserve la sélection courante, indépendamment du budget de diagnostic.

3:00–4:00 — vérifier le vrai navigateur. « Le carnet nous montre un historique interne. Ici, dans le projet, l'URL et le bouton Retour doivent restaurer l'état. [Ralentir.] On vérifie donc ce parcours dans son navigateur cible. » Faire un aller-retour et consigner un éventuel écart.

4:00–5:00 — transférer. « Si cette vue devient une galerie, que faut-il conserver lorsque tu ouvres une image puis reviens ? Et si la page est figée, quelle horloge doit rester en place ? » Accepter schéma ou code annoté, avec un essai nouveau avant une conclusion de compréhension.

Repérer et travailler une difficulté

La confusion à examiner est : **mélanger l'état du geste, l'état de la vue et l'historique global du navigateur**. Demander un exemple précis, faire retrouver une seule donnée, puis choisir une expérience plus petite si nécessaire. Une réponse fluide de l'assistant peut devenir une hypothèse de travail ; l'élève la relie au fichier avant de la conserver.

Retrouver le symbole `createInteractionFlow` et appliquer le cadre d'intervention commun à cette difficulté.

Transfert et retour humain

Proposer **une galerie qui revient à la bonne sélection**. L'élève peut répondre par texte, schéma ou code annoté, puis essayer la proposition. Relier le retour humain au scénario de transfert effectivement examiné.

Fiche enseignant 8 — Assistant, preuves et capacités

But de la revue : L'élève attribue correctement les preuves, borne la lecture et explique sa contribution dans le projet assemblé.

Préparation : ouvrir le rendu sélectionné, le notebook et register-capabilities.js. Le corrigé séparé est une piste à examiner après la tentative. Retrouver la prédiction et la version réellement choisie avant de comparer.

Une activité avec l'élève et son assistant

L'élève examine la trace préparée, montre l'objection liée à la permission ou à la révision et définit son contenu partageable. Pendant la séance, il retrouve une vraie modification dans chacun des huit exports assemblés.

L'assistant peut expliquer ou fournir un exemple complet. L'élève choisit ensuite sa modification, nomme l'aide reçue et montre un résultat. La question à poser est : « **Quelle preuve distingue une trace préparée, un appel réel et une revue humaine ?** »

Comparer **borne de snapshot : 12000 → 6000 caractères JSON**, en conservant **permission, révision, vérification et nettoyage**. Le rendu sélectionné peut prendre la forme de périmètre de capacité, objection, code choisi et lien vers les exports. La limite à garder visible est : **le notebook ne réalise pas l'appel natif ; student_get_learning_snapshot diffère du registre Orbit de vingt-cinq outils**.

Déroulé et script de séance

Durée	Action avec l'enseignant
10 min	Examiner la trace, le code exporté et les permissions prévues
15 min	Clarifier citation, portée, révision et autorité
25 min	Utiliser les vrais outils d'Orbit, puis raccorder vos huit productions
10 min	Transférer une capacité bornée à votre projet et expliquer le socle obtenu

La découverte de fin de leçon

Vous avez conservé huit briques. **Elles peuvent maintenant former un socle de frontend** : geste et état, carte, scène, inertie, transition, particules, parcours et capacité.

Pendant les 25 minutes de construction du webinaire, ouvrir la partie finale du notebook 8. Le parcours comporte trois étapes distinctes :

1. **Charger — cellule 13.** Choisir vos huit ZIP réels avec le sélecteur officiel `files.upload()`. Une annulation laisse la sélection vide et permet de relancer la cellule. Si le dialogue reste indisponible, téléverser les mêmes ZIP par le panneau **Fichiers** → **Importer dans l'espace de stockage de la session** de Colab. Activer alors `USE_RUNTIME_FILE_SELECTION = True` et écrire exactement leurs huit noms dans `RUNTIME_SELECTED_EXPORT_FILENAMES`. Cette alternative lit les fichiers explicitement nommés dans `/content` ; elle ne recherche ni ne télécharge de rendus à votre place. Une session Colab est temporaire : conserver les originaux et télécharger le résultat avant sa fermeture.
2. **Examiner — cellule 14.** Vérifier les huit lignes : module, nom du fichier choisi, tentative, formats déclarés du résultat et du manifeste, taille et SHA-256. Chaque module doit apparaître une seule fois. Un fichier absent, un doublon ou une empreinte différente appelle une correction et un nouvel examen.
3. **Assembler — cellule 15.** Copier l'empreinte de la sélection examinée dans `ASSEMBLY_REVIEWED_SELECTION_SHA256`, puis exécuter la cellule. Une confirmation vide ou ancienne laisse l'assemblage bloqué. Télécharger `orbit-mon-frontend-<empreinte12>.zip` et conserver sa provenance. Le statut `ASSEMBLED_NOT_BUILT` indique que le ZIP est préparé ; il ne déclare pas une compilation.

Le raccordement fourni contrôle les manifestes et conserve vos fichiers. Il ne comble pas une absence avec un corrigé caché. Si un rendu doit être remplacé, choisir et expliquer sa nouvelle version, puis reprendre **charger** → **examiner** → **assembler**. Ce raccordement reste inclus dans la construction du webinaire ; il n'ajoute aucune heure au cours.

Le code d'intégration est fourni et attribué au cours dans `assembly/`. Vos modifications sont dans les dossiers `src/learning/module-N/`. L'assemblage du ZIP ne compile pas Astro : la compilation et le parcours du projet sont vérifiés séparément. Une modification hors du contrat appelle une adaptation expliquée ; elle ne justifie pas une substitution silencieuse du travail.

Script enseignant — séquence de 5 minutes

Les intonations sont proposées ; elles ne constituent pas un clonage de voix.

0:00–1:00 — choisir ce qui est partagé. « Je veux que ton assistant puisse t'aider sur un contenu que tu choisis. [Pause.] Quelle fiche lui donnes-tu, et quelle révision ? Montrons ces deux conditions dans le code. » Laisser l'élève borner le périmètre avant l'appel.

1:00–2:00 — lire la trace. « Voici une réponse préparée qui dit `READY`. [Accentuer : *préparée*.] Ce mot peut-il prouver que le navigateur a réellement exécuté l'outil ? » Attendre la distinction puis examiner les champs incohérents avec la permission ou la révision.

2:00–3:00 — vérifier l'autorité. « Un agent peut expliquer, calculer et proposer. Une revue humaine demande une décision réellement humaine. [Ralentir.] Regardons aussi le passage et les conditions avant d'accepter la conclusion. » Demander une objection précise plutôt qu'un sentiment général de confiance.

3:00–4:00 — découvrir le raccordement. « Tu as gardé tes huit productions. Maintenant, on va les raccorder. Ce sont tes exports qui entrent dans le projet ; le code d'assemblage est fourni. » Montrer un chemin réellement importé et laisser l'élève retrouver une modification.

4:00–5:00 — choisir la suite. « Ce socle appartient maintenant à ton projet. Quel usage veux-tu en faire ? Et quelle capacité utile donneras-tu à ton assistant, avec un partage précis ? » Faire expliquer un nouvel usage et un contrôle. Garder les incertitudes pour la revue finale.

Repérer et travailler une difficulté

La confusion à examiner est : **prendre READY, une empreinte ou une proposition pour une approbation humaine**. Demander un exemple précis, faire retrouver une seule donnée, puis choisir une expérience plus petite si nécessaire. Une réponse fluide de l'assistant peut devenir une hypothèse de travail ; l'élève la relie au fichier avant de la conserver.

Retrouver le symbole `registerLearningCapability` et appliquer le cadre d'intervention commun à cette difficulté.

Transfert et retour humain

Proposer **une capacité bornée utile à son propre projet**. L'élève peut répondre par texte, schéma ou code annoté, puis essayer la proposition. Relier le retour humain au scénario de transfert effectivement examiné.

Fiche enseignant projet — cadrage et clôture

Une heure de cadrage

	Durée	Action	Trace
	10 min	Décrire la personne, son besoin et l'action principale	Intention en trois phrases
	15 min	Comparer deux pistes et choisir un parcours court	Choix, raison et limite
	20 min	Définir contenu, geste, contribution 3D, HTML et capacité	Carte du projet et critères
	15 min	Choisir l'expérience, la remise et les points de revue	Plan de six heures solo

L'idée appartient à l'élève. La contribution 3D sert un usage expliqué et le contenu conserve une alternative HTML. Les briques fournies, les modifications et le raccordement sont attribués séparément.

Six heures solo, suivies de la clôture

Distribuer 1 h de préparation, 2 h d'adaptation, 2 h d'intégration/expérience et 1 h de préparation de remise. Dans le calendrier, cela correspond aux derniers modules. La séance de clôture suit ce travail ; aucune heure accompagnée supplémentaire n'est ajoutée.

Une heure de clôture

	Durée	Action	Examen
	10 min	Montrer besoin et parcours	Usage explicite
	20 min	Démontrer geste, sélection, retour, statique et HTML	Comportement observé
	15 min	Expliquer une modification et rejouer une expérience	Données, paramètres et limite
	10 min	Appliquer une notion à un autre contexte	Transfert nouveau

Durée	Action	Examen
5 min	Conserver acquis observés et prochaine question	Bilan sélectionné

Script enseignant — séquence de 5 minutes

Les pauses et intonations sont des propositions de lecture.

0:00–1:00. « Je veux partir de ton intention. À qui sert ce frontend, et quelle action cette personne doit-elle pouvoir accomplir ? [Pause.] Montre-moi ce parcours avant de parler des bibliothèques. »

1:00–2:00. « Maintenant, choisis une modification qui t'appartient. Où est-elle dans le fichier ? Qu'as-tu gardé du modèle, et quelle aide as-tu reçue ? » Laisser retrouver le code plutôt que réciter une explication générale.

2:00–3:00. « Prenons ton expérience. [Ralentir.] Quelle variable as-tu changée ? Quelles conditions as-tu conservées ? Montre-moi l'observation et une limite. » Examiner un résultat concret ; une observation insuffisante conserve son statut.

3:00–4:00. « On va essayer le même contenu avec moins de mouvement et au clavier. Est-ce que la personne retrouve toujours son information ? » Laisser réaliser le parcours et consigner un éventuel défaut, sans l'effacer du bilan.

4:00–5:00. « Tu pourrais réutiliser cette notion ailleurs. Propose un exemple qui n'est pas cette scène. Quelle donnée ou quelle règle garderais-tu ? » Laisser reformuler puis définir une prochaine vérification utile.

Revue de projet

Demander de retrouver une modification réelle dans les exports, d'expliquer un événement qui change l'état et de montrer la reprise. Examiner une expérience comparable plutôt qu'une impression seule. Donner un exemple nouveau de carte, de galerie ou de capteur pour le transfert.

Conserver une grille : intention, attribution, état, accessibilité, expérience, capacité bornée, reprise et transfert. Chaque ligne reçoit le scénario, le constat et sa limite. L'enseignant garde son retour séparé de la proposition de l'assistant et de l'empreinte des fichiers. La publication publique reste une action distincte.

Conclusion générale — une production que l'on peut expliquer

Le parcours laisse une notion, une expérience et un fichier par module. Le projet final relie les productions au besoin choisi. L'enseignant examine les résultats, les limites et un transfert. L'assistant aide à avancer ; l'élève peut montrer ce qu'il a compris, ce qu'il conserve et ce qu'il veut vérifier ensuite.

Les scripts et les critères de ce manuel préparent cet examen. Les observations d'apprentissage appartiennent aux séances réellement effectuées.

Annexes

A — Remettre et reprendre un travail

Avant l'export, vérifier le module, la tentative, le paramètre conservé, les conditions et la question ouverte. Choisir les fichiers et observations à remettre. Conserver le notebook et le ZIP dans son espace personnel.

À l'import, Orbit vérifie les fichiers et les empreintes de l'archive compatible. Le contenu importé reste déclaré externe. Une nouvelle version conserve l'ancienne ; son partage est choisi séparément. Calculer une empreinte permet de comparer le contenu, avec une explication distincte pour l'exécution et la compréhension.

Si la sauvegarde locale échoue, lire le message durable et exporter la session avant de fermer. Lors d'une reprise de session, choisir à nouveau les permissions et les éléments partagés. Un ancien résultat de l'assistant ne doit pas écraser une révision nouvelle.

Pour Colab, distinguer relecture dans le même runtime, namespace neuf et redémarrage réel. Retrouver les fichiers sauvegardés après un redémarrage avant de relancer l'expérience. Un ZIP avec une tentative réutilisée et un contenu différent demande une correction de la tentative, pas une importation ambiguë.

À la fin du Module 8, charger les huit exports, examiner leurs versions déclarées et leurs empreintes, puis confirmer l'empreinte de cette sélection avant d'assembler. Les cellules 13, 14 et 15 conservent ces trois étapes. Le sélecteur officiel permet l'annulation et la reprise. Si son dialogue reste indisponible, le panneau Fichiers de Colab permet de téléverser les mêmes ZIP ; l'élève active ensuite la lecture des huit noms qu'il a explicitement déclarés. L'assemblage téléchargé reste distinct de sa compilation et de la compréhension examinée pendant la séance.

B — Les vingt-cinq outils dans le contexte pédagogique

Fonction	Outils
Découverte et méthode	<code>orbit_get_capabilities</code> , <code>orbit_get_research_protocol</code>
Context	<code>orbit_sanity_initial_context</code> , <code>orbit_sanity_read_entries</code>
Question et mission de recherche	<code>orbit_get_research_request</code> , <code>orbit_get_mission_summary</code> , <code>orbit_list_research_points</code>
Sources	<code>orbit_search_sources</code> , <code>orbit_read_source_record</code>
Proposition de recherche	<code>orbit_present_research</code>

Fonction	Outils
Preuves et relations	<code>orbit_classify_evidence</code> , <code>orbit_compare_claims</code> , <code>orbit_find_relations</code> , <code>orbit_resolve_hold</code> , <code>orbit_trace_impact</code>
Mission et méthode pédagogique	<code>orbit_get_learning_mission</code> , <code>orbit_get_learning_protocol</code>
Activité et expérience	<code>orbit_plan_learning_activity</code> , <code>orbit_prepare_experiment</code>
Artefact et aide	<code>orbit_read_learning_artifact</code> , <code>orbit_get_learning_support</code>
Revue et transfert	<code>orbit_check_understanding</code> , <code>orbit_prepare_transfer</code>
Proposition et journal	<code>orbit_present_learning_work</code> , <code>orbit_get_learning_journal</code>

Les commandes humaines restent utilisables lorsque WebMCP est indisponible. Le registre réellement découvert et la permission courante gouvernent l'appel. Les références privées exigent la sélection, le contexte et la révision appropriés. PRESENTED indique un dépôt pour revue. READY indique une ressource ou un calcul disponible. Les états CONSENT_REQUIRED, STALE_REVISION, NOT_FOUND et UNAVAILABLE indiquent la condition à traiter.

Le contexte pédagogique possède un registre par document. Son propriétaire de montage est distinct du dossier conservé : `registerFormationTools` attend la fermeture de l'ancien enregistrement avant de reprendre les noms natifs; `unregisterFormationTools` ferme seulement l'enregistrement du propriétaire concerné. Ainsi, le nettoyage tardif d'une ancienne vue garde intacte sa vue successeure. Le changement de module, la révocation et la fermeture invalident les accès privés. Ces responsabilités décrivent le code partagé; leur preuve navigateur appartient au reçu du même payload.

Les deux outils Context lisent le corpus du cours par **GET** sur `/api/v1/course-context/outline` et `/api/v1/course-context/entries`, avec les credentials du navigateur omis. Le transport ne transmet pas le journal personnel. Le serveur borne ces lectures publiques et garde son activation explicite; une passerelle indisponible produit UNAVAILABLE. Ce transport et la vraie session Sanity du Studio sont deux capacités à qualifier séparément.

Les trois moteurs sont baseline, n et p, seuls ou par paires sur les mêmes données. Une session neuve n'en exécute aucun implicitement. Leurs résultats restent séparés ; une moyenne ou un vote ne désigne pas un gagnant. T et F peuvent coexister, I peut subsister et ces ensembles ne constituent pas des probabilités. Une date différente demande une relation de version justifiée ; deux attributs manquants restent inconnus.

C — Installer le plugin dans son Studio

Le plugin apporte les vues Orbit · Lab et Orbit · Projects à un Studio déjà détenu par l'élève. Le Studio hôte fournit projet, dataset, session et droits. Le plugin contient le cœur partagé ; React et Sanity proviennent de l'hôte. Le guide d'installation maintenu donne l'archive et les versions examinées.

Les travaux privés commencent en mémoire ou dans le stockage local autorisé. La publication Sanity est une action humaine : sélectionner les artefacts publiables, examiner le JSON exact, la destination et la révision, puis effectuer la commande avec les droits réels. Le journal privé et les

permissions ne sont pas ce contenu publié. Une compilation du plugin et un parcours authentifié sont des preuves distinctes.

Le second hôte préparé sous `/formation/studio/` conserve le projet et le dataset de l'équipe, avec un autre point de montage. La session et les droits appartiennent toujours à Sanity. Deux routes sur la même origine gardent le même périmètre d'utilisateur; elles ne créent pas une séparation de confidentialité. Installer le plugin dans le Studio personnel de l'élève utilise son propre projet, dataset et session. Le transport public du cours et cette session authentifiée gardent des autorisations distinctes.

D — Carte du code

La carte détaillée suit le tableau des modules ci-dessous. Les fichiers de raccordement sont fournis et attribués au cours. Les dossiers `src/learning/module-N/` contiennent les exports réellement sélectionnés. L'assemblage ne remplace pas une brique absente par une correction cachée.

Module	Fichier et symbole	Paramètre de l'essai
1	<code>interaction-state.js · create</code> Interaction	<code>keyboardStep</code> : 0.04 → 0.08
2	<code>exploration-card.js · create</code> Cards	<code>descriptionPrefix</code> : Mon observation → Ma décision expliquée
3	<code>scene-controller.js · create</code> SceneController	<code>cameraDistance</code> : 5 → 7
4	<code>inertia.js · createInertia</code>	<code>damping</code> : 0.65 → 1.3
5	<code>transitions.js · create</code> Transition	<code>damping</code> : 12 → 5
6	<code>particle-layer.js · create</code> ParticleLayer	<code>count</code> : 64 → 128, puis restauration à 64
7	<code>interaction-flow.js · create</code> InteractionFlow	<code>historyLimit</code> : 32 → 4
8	<code>register-capabilities.js ·</code> <code>registerLearning</code> Capability	borne de snapshot : 12000 → 6000 caractères JSON
Raccordement fourni	<code>student-frontend.js,</code> <code>merge_modules.py</code>	Les huit ZIP réels, puis compilation et parcours séparés

E — Glossaire / Glossary

Notion	Explication liée au cours
État	Données conservées pour décrire un geste, une sélection ou une vue
Événement	Signal reçu par le programme, comme une pression, un déplacement ou un retour navigateur
Capture du pointeur	Acheminement des événements vers l'élément qui tient le geste, avec un identifiant actif
Rendu	Présentation calculée à partir de l'état

Notion	Explication liée au cours
Identifiant stable	Valeur reliant un même élément à sa carte, sa scène et sa fiche
Coordonnée normalisée	Position relative à la zone ; ici entre 0 et 1
Caméra et projection	Point de vue et transformation de la scène vers une image
Maillage	Géométrie et matériau regroupés pour représenter un objet
Matériau	Paramètres qui décrivent la réaction visuelle d'une surface
Raycasting	Recherche d'un objet rencontré par un rayon de sélection
Buffer	Zone de données réutilisable, par exemple pour positions de particules
Inertie	Conservation d'une vitesse après relâchement dans ce modèle d'interface
Amortissement	Terme réduisant la vitesse dans le modèle fourni
Interpolation	Règle reliant une valeur présente à une cible
Ressort	Évolution conservant aussi une vitesse et pouvant dépasser la cible
Boucle de rendu	Cadence qui met à jour les données puis demande leur dessin
Historique borné	Liste diagnostique limitée à un nombre d'entrées
Nettoyage	Fermeture des événements et libération des ressources possédées
Révision	Version du travail sur laquelle une proposition ou une permission s'applique
Empreinte	Valeur permettant de comparer l'identité d'un contenu
Provenance	Origine, référence, version et statut de lecture d'un élément
Portée	Conditions d'application d'une affirmation
HOLD	Suspension justifiée indiquant l'information attendue et la condition de reprise
Transfert	Réemploi d'une notion dans une situation nouvelle

F — Symboles et unités

Symbole ou champ	Sens dans les fichiers du cours
x, y	Position normalisée, sans unité de longueur physique
v_x, v_y	Variation de position normalisée par seconde
dt	Durée d'une étape, en secondes ; le modèle borne les grandes durées
k / damping	Coefficient d'amortissement ; dans la loi exponentielle, $k \times dt$ est sans dimension

Symbole ou champ	Sens dans les fichiers du cours
value, target	Valeur d'échelle présente et cible dans la transition
velocity	Variation de l'échelle par seconde
count, age, lifetime	Population, âge et durée de vie d'une particule de démonstration
T / I / F	Soutiens, raisons d'indétermination et réfutations indépendantes pour une affirmation

Les nombres servent ici aux mécanismes d'interface. Une graine répétable, une sphère ou une trajectoire ne donnent pas une validation physique du modèle.

G — Index des notions / Concept Index

- Accessibilité : Modules 1, 2, 3, 5 ; fiches enseignant ; annexes A et C.
- Assistant, autorité et attribution : Module 8 ; partie II ; annexes A, B et C.
- Caméra, coordonnées et sélection : Module 3 ; carte du code ; glossaire.
- État et événements : Modules 1 et 7 ; fiches enseignant 1 et 7.
- Empreintes, preuve et révision : Module 8 ; qualification ; annexes A et B.
- Inertie, vitesse et durée : Module 4 ; symboles et unités.
- Interpolation et ressort : Module 5 ; fiche enseignant 5.
- Particules et nettoyage : Modules 3 et 6 ; fiche enseignant 6.
- Projet, remise et reprise : projet personnel ; fiche enseignant projet ; annexe A.
- Transfert : critères de chaque module ; clôture du projet.

H — Sources et références

Les fichiers du cours et les sources primaires cités dans chaque chapitre sont recensés dans `source-map.json`. La carte conserve leurs empreintes au moment de la composition, leur rôle et leur état. Les sorties générées utilisent ce contenu commun. Les horaires et missions restent gouvernés par le catalogue ; la qualification reste gouvernée par le reçu de livraison.

- Catalogue des missions.
- Contrat du cours.
- Contrat des outils WebMCP.
- Installation du plugin Studio.
- Export et assemblage.
- FAQ officielle Colab.
- Documentation Astro.
- Manuel Three.js.
- API WebMCP Chrome.
- Sanity Context.

Mot de clôture

L'élève choisit une production qu'il peut montrer et une notion qu'il peut expliquer dans un nouvel exemple. L'enseignant conserve ce qui a été effectivement examiné et prépare la question suivante. Le manuel et ses documents web soutiennent ce travail concret ; la revue humaine en garde la responsabilité.

Empreintes intégrales de lecture

Les tableaux et paragraphes de cette édition abrègent les identifiants longs pour garder leur lecture dans la page. Les valeurs intégrales ci-dessous sont recopiées sans modification depuis le manuscrit maintenu; les reçus liés précisent leur objet et leur scénario.

Valeurs intégrales, numérotées par ordre de première apparition dans le manuscrit :

01 7ae43df9d3c7f44e39a2b545d28ff807660b85615551b2fca1a55913b339c7d6
02 28813ecbf712f4fb717aaa3f93444c7c7e14fbd737f2d3e3ac1ed6ca28f72531
03 98a0c6594d176b3cb4fe7b921fd93e945e51d96f3fba3019d86037585062ed87
04 689494986b4b6a9b02b5a22a8beb81e8de7cb7388a55ad20a122743d3de7966e
05 ea0f93eff48e9dd6a07283f46067eafdc317b352e43409efac8e5e61d4992b90
06 f2b0dc6897fae7b56e36836e4019eff06e098bf14d35668fa1df84ec6ff9e973
07 7e6c739c0883e39aca491e6f6280cc353daf6745017a3bea5ce690746e4d67a5
08 4b06da41e603d63acd33bbf75995a75d824d1bc6
09 827528418863211e76307d3e3dbd2a21d17d2864bde0e59f4d8295948897f3ce
10 96acc0ddfd3728426980df39d2a1ccda07cc855eb95f950a552de776a5b2aed
11 b202088c8f092630676273c7cdf97d89302983866f70f5b8df3305486d7c3335
12 8d80562e069f26bb07ec097ad5763676548bf125d7ab06ce5529f8d590bd1654
13 141466603afaca979af3e8ee011391c9516de2a723c52809bdca86f0dbb0b1ba
14 423e48783d884b71827bef1e4aae380a3e15ca93735ef1a89423cf54e8a0edd8
15 e4215188e3f702b8707d3992aaa0f917bbdca00e58118a0f6953d98697412938
16 ed94c13524c078f84c44f074a3034d10850f81470a80603638c3404663b4ba06
17 d8ff5bd2c54474cca831101f9992d4b92af273370c6f53dee07cd7b639c2d2ce
18 3acb942558e5dc6e42104494238a6191af12c274d607f184843611be3166807b
19 6b85b3780c50e10cb02a4e75e2ef553b15989f2487cef500ed15b6d572cc0f45
20 1426f7985237d66f60736b2b55e96e8b47aa6d9736e161ce25bef40b461b031b